# Froth

## Machine Learning study notes

Learning by building: 47 notes that came out of a real project

From what an embedding is to why 10,921 papers fit on screen and 1,772 do not.
Every note came from something that actually happened, mistakes included.

Froth project: semantic mapping of scientific literature

Built on 2026-08-19

# Contents

# Note 00: The project map (read this first)

Froth project · learning Machine Learning by building it

## What we are building, and why

**Froth** is a tool that, given a scientific topic, downloads hundreds of papers, "understands" them by their content and draws a *map of subtopics*: bubbles of papers grouped by meaning, connections between groups, and empty areas (the *gaps* where you can contribute something new in your thesis).

The project has a double goal, and both weigh the same:

1. **Actually learn Machine Learning**, piece by piece, by building it.
2. **Have a useful product** for complete literature reviews, not the classic "5 papers that confirm what I already thought".

Concept: pipeline

The whole system is a **chain of 5 steps** where the output of one is the input of the next:
`[1] PULL` → `[2] EMBED` → `[3] REDUCE` → `[4] CLUSTER` → `[5] VISUALIZE`

1. **Pull**: download papers (title, abstract, authors...) from public databases.
2. **Embed**: turn each abstract into a *vector* (a list of numbers that captures its meaning).
3. **Reduce**: flatten those ∼768-dimensional vectors down to 2 so they can be drawn.
4. **Cluster**: group the nearby points = subtopics discovered automatically.
5. **Visualize**: bubbles, mind map, gap detection, review draft.

Each step is, at the same time, an ML concept you are going to master.

## The folder map (what each thing is, and in what order to look)

Everything lives inside `Documents\Froth\`. **Golden rule:** the folders *with a number* are yours (go in); the ones *without a number* are the machinery (do not touch). The full cheat sheet is in the file `0_LEEME_PRIMERO.md` at the root.

**Yours (numbered):**

1. `1_Apuntes/`: **your library** (these PDFs). Read them in order: 00, 01, 02...
2. `2_Datos/`: the data. `raw/` (raw), `processed/` (clean tables), `embeddings/` (vectors). It fills up on its own when you run the pipeline.
3. `3_Resultados/`: the visible outputs, maps and reviews.
4. `4_Notebooks/`: experimentation notebooks (Jupyter), when they arrive.
5. `5_Bitacoras/`: decisions taken and your learning diary.

**Machinery (no number, do not touch):**

- `froth/`: **the source code** (the engine). One file per pipeline step: `config.py` (settings) → `pull.py` → `embed.py` → `cluster.py` → `graph.py` → `gaps.py` → `visualize.py`.
- `.venv/`: the bubble holding Python and its libraries (see Note 01).

- `tools/`: the compiler that turns these notes into PDFs.

**Watch out / common mistake**

Why does `froth/` **not** carry a number, if we said everything should be ordered? Because Python imports code by its name (`from froth import pull`) and a name starting with a number breaks the language. The *order* of the pipeline does not live in the folder name but inside the code (`froth/__init__.py`). The notes and your own folders, on the other hand, are numbered because nobody imports them: they are only read.

## The 7 phases at a glance

**F0 Foundations**: Python environment, Git, structure. *(nearly closed)*

**F1 Pull**: download 200 to 500 papers on a topic. *(first version already works: 126 papers)*

**F2 Embeddings**: text → vectors; semantic search. *(next)*

**F3 Clusters**: the subtopic bubbles (UMAP + HDBSCAN + BERTopic).

**F4 Mind map**: a graph of relations between papers and topics.

**F5 Gaps + draft**: empty areas of the map and a structured review text.

**F6 Deep learning**: PyTorch, transformers, fine-tuning your own model.

## How to use these notes

Every time a new skill or concept shows up in the project, a numbered PDF note is generated and saved in `apuntes/`. You just go in there and read in order. Each note has coloured boxes: **Concept** (the idea), **Try it yourself** (commands for you to type) and **Watch out** (typical mistakes).

**In one sentence**

Froth is a 5-step pipeline that turns “a topic” into “a map of the literature”; this note is your general blueprint: folders, phases and reading order.

# Note 01: Python, the engine; and the virtual environment, the toolbox

Froth project · learning Machine Learning by building it

## Python: what it really is

The `.py` files in the `froth/` folder are plain text: written recipes. **Python** is the program that knows how to read and run them (the "engine"). That is why the first step of the project was installing it: with no engine, the recipes are just paper.

Your machine ended up with **Python 3.14**. When you type `python file.py` in a terminal, you are saying: "engine, run this recipe".

**Concept: library (package)**

Almost nothing is programmed from scratch. A **library** is packaged code that other people wrote and that you install and reuse: `requests` (talking to the internet), `pandas` (data tables), and so on. They are installed with the `pip` tool, which comes with Python. Think of them as apps you install into the engine.

## The problem the virtual environment solves

Imagine two projects on your PC: one needs `pandas` version 2 and the other version 3. If you install everything "into the system", they step on each other and something breaks. The standard solution:

**Concept: virtual environment (venv)**

A **bubble folder** (in our case `.venv/` inside the project) holding a private copy of Python and its libraries, exclusive to this project. What you install inside touches nothing outside, and the other way round.
It was created with: `python -m venv .venv`
And since then, to run anything in the project we use the Python *from the bubble*: `.venv\Scripts\python.exe`. That odd path at the start of every command means exactly that: "use THIS project's engine".

## requirements.txt: the shopping list

The `requirements.txt` file lists the libraries the project needs, with a comment saying what each one is for. Anyone (you on another PC, a colleague) can recreate the environment with a single order:

```
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

In Froth we install them *by phase*: today only the Phase 1 ones (`requests`, `pandas`, `pyarrow`, `truststore`). The heavy ML ones will arrive when their turn comes.

**Watch out / common mistake**

If a command tells you `ModuleNotFoundError: No module named 'pandas'`, it almost always means you ran it with the **system** Python instead of the bubble's. Check that the command starts with `.venv\Scripts\python.exe`.

**Try it yourself (hands on the keyboard)**

Open PowerShell (Windows key → type `powershell` → Enter) and:

1. `cd "C:\Users\you\Documents\Froth"`
2. `.venv\Scripts\python.exe --version` (it should say 3.14)
3. `.venv\Scripts\python.exe -m pip list` (the libraries in your bubble)

**In one sentence**

Python runs the code; the `.venv` is the project's private bubble with its libraries; `requirements.txt` is the list for rebuilding it anywhere.

# Note 02: APIs and JSON: how you ask a server for data

Froth project · learning Machine Learning by building it

## The idea in 30 seconds

Your PC (the *client*) sends a request over the internet to another computer (the *server*) and that one answers with data. That is all. An **API** (*Application Programming Interface*) is simply the official counter a service offers so that *programs* (not people with a browser) can ask it for things.

In Froth we use the **OpenAlex** API, a free catalogue with ∼250 million academic works.

**Concept: HTTP GET request**

A request is written as a URL with **parameters** after the `?` sign:

```
https://api.openalex.org/works?search=lepidolite flotation&per_page=25
```

It reads: "from the `works` catalogue, search me `lepidolite flotation`, give me 25 per page". In Python, the `requests` library builds and sends this for you:

```
r = requests.get(url, params={...})
```

**Concept: JSON**

The server answers in **JSON**: a text format that looks just like Python dictionaries and lists (`{"title": "...", "year": 2021}`). That is why `r.json()` turns it straight into Python objects you can already work with. JSON is *the* data exchange language of the internet.

## Details of a polite citizen

Public APIs are a shared resource. Three habits our `pull.py` already practises:

1. **Identify yourself** (the *polite pool*): we send our email in the `mailto` parameter. OpenAlex gives better service to those who show their face.
2. **Do not hammer it**: between one search and the next we wait 0.3 s (`time.sleep(0.3)`). Thousands of requests per second take services down.
3. **Ask only for what you need**: the `select` parameter lists the fields we want (title, year, authors...). Less data = faster for everyone.

**Concept: the inverted abstract trick**

OpenAlex does not store the abstract as running text but as an *inverted index*: a dictionary `{word: [positions where it appears]}`. It is a real quirk of this API (they do it for legal and efficiency reasons). Our `_reconstruct_abstract()` function puts it back together: it places each word at its position and joins them. First example in the project of the fact that **real data arrives "dirty"** and has to be worked on.

**Try it yourself (hands on the keyboard)**

Paste this URL into your browser (an API answers there too) and look at the raw JSON:

```
https://api.openalex.org/works?search=lepidolite%20flotation&per_page=2
```

Look for the `title`, `publication_year` and `abstract_inverted_index` fields in the response.

**In one sentence**

An API is a counter for programs: you send a URL with parameters, they answer JSON, and `requests` + `r.json()` turn it into Python dictionaries.

# Note 03: When the internet says no: SSL, error codes and API keys

Froth project · learning Machine Learning by building it

Today we ran into two different errors one after the other. It is worth gold to tell them apart, because one was *your machine's* and the other *the server's*, and they are fixed in opposite ways.

## Error 1: the padlock (SSL / certificates)

**Concept: certificates and HTTPS**

When your PC connects to `https://api.openalex.org`, before sending anything it checks the server's **certificate**: its digital ID card, signed by trusted authorities. That is how you know you are talking to the real OpenAlex and not to an impostor.
Python carries its own list of trusted authorities. On your network (corporate, with an antivirus that inspects traffic), the certificate that arrived was not on that list, so Python refused: `CERTIFICATE_VERIFY_FAILED`. Your browser did work, because it uses the *Windows* list, which does know that certificate.
**Fix:** the `truststore` library tells Python "use the Windows list". We turn it on once in `froth/__init__.py` and it holds for the whole project.

## Error 2: the closed door (code 503)

**Concept: HTTP status codes**

Every HTTP response carries a 3-digit number summing up how it went:

- **2xx**: fine (`200 OK`).
- **4xx**: the error is *yours*. `404` does not exist, `401` no permission, `429` you are asking too fast.
- **5xx**: the error is *the server's*. `500` it broke inside, `503` "unavailable" (saturated or under maintenance).

Our `503` came with a clear message: "anonymous search limited by load; use a free API key". There was nothing to fix in the code: it was their door, closed to anonymous callers.

**Watch out / common mistake**

A mental rule for life: **SSL/certificates → problem on your side** (network, Python, certificates). **5xx → problem on their side** (wait, identify yourself, or change service). Retrying in a loop without understanding which of the two it is wastes your time.

## The API key: your VIP card (and why it is a secret)

**Concept: API key**

An **API key** is a string of text that identifies your requests to the service. With it OpenAlex gives you stable access (1 USD/day of free usage, around 1000 searches, more than enough). It is sent as one more parameter: `api_key=...`.
Because it identifies *you*, it is a **secret**: whoever has it spends your quota in your name. That is why:

- It is **never** written inside the source code.
- It lives in the `.openalex_key` file at the project root (a single line).
- That file is in `.gitignore`: the list of things Git (version control) is forbidden to upload anywhere. If one day you publish the code on GitHub, your key does not travel with it.

`config.py` reads it at startup (first it looks for the `OPENALEX_API_KEY` environment variable; failing that, the file) and `pull.py` adds it to every request.

**Try it yourself (hands on the keyboard)**

Check for yourself that the key is where it should be and out of Git's reach:

1. Open `.gitignore` (with Notepad) and find the `.openalex_key` line.
2. In PowerShell, inside the project folder: `Get-Content .openalex_key` (you will see your key, only you).

**In one sentence**

SSL failed on your machine (fixed with truststore); the 503 was the saturated server turning away anonymous callers (fixed with an API key); and an API key is a secret that lives outside the code and outside Git.

# Note 04: Data tables: pandas and the parquet file

Froth project · learning Machine Learning by building it

## What we got today

The project's first haul: **126 real papers** on your topic, saved in `data/processed/papers.parquet`. This note explains what that "table" is and why it is saved that way.

**Concept: DataFrame (pandas)**

`pandas` is THE table library in Python, and its star object is the **DataFrame**: a table with rows and columns, like an Excel sheet but driven by code. Ours has one row per paper and these columns:

| | |
|---|---|
| `id`, `doi` | unique identifiers of the paper |
| `title`, `year` | title and year |
| `citations` | how many times it has been cited |
| `authors` | authors (text "A; B; C") |
| `abstract` | the summary, **the raw material of the ML** |
| `query` | which search it turned up in (for debugging) |

With a DataFrame you do in one line things that would be painful in Excel: `df.drop_duplicates(subset="id")` removes repeated papers; `df[df["abstract"].str.len() > 20]` filters out the ones with no abstract; `df.sort_values("citations")` sorts by citations.

## Cleaning matters more than it looks

`pull.py` does not save what arrives as it arrives. It does three cleanups:

1. **Deduplicate**: the same paper can show up in several searches ("lepidolite flotation" and "lithium mica flotation" bring overlapping results).
2. **Filter out the abstract-less**: with no summary there is no meaning to vectorise.
3. **Rebuild the abstract** (see Note 02).

**Watch out / common mistake**

A few **off-topic** papers slipped into our 126 (for example one about telecommunications). Why? An acronym collision: we searched "LCT" (a type of pegmatite) and it also exists in other fields. We are not fixing it by hand: in Phases 2 and 3 the ML itself will set them aside, because their *vectors* will land far from the flotation ones. Real data always carries noise; the system has to tolerate it.

**Concept: parquet vs. CSV**

We could have saved the table as `.csv` (plain comma-separated text). We chose **parquet** because:

- **It compresses**: it takes several times less space.
- **It stores by column**: reading only `year` and `citations` from 100 000 papers does not force you to read the abstracts.
- **It keeps the types**: in CSV everything comes back as text and you have to re-guess what was a number; parquet remembers that `year` is an integer.

It is the standard format in modern data science. The `pyarrow` library is the one that knows how to read and write it; pandas uses it underneath with `df.to_parquet()` and `pd.read_parquet()`.

**Try it yourself (hands on the keyboard)**

Look at your haul with your own hands. In PowerShell, inside the project folder, open an interactive Python with `.venv\Scripts\python.exe` and type line by line:

1. `import pandas as pd`
2. `df = pd.read_parquet("data/processed/papers.parquet")`
3. `len(df)` (how many papers?)
4. `df.columns` (the columns)
5. `df.sort_values("citations", ascending=False).head(3)[["title","year"]]`
6. `exit()` to leave.

You have just done your first data analysis in pandas.

**In one sentence**

A DataFrame is a programmable table; we clean duplicates and abstract-less rows; and parquet is the compressed, columnar, type-preserving format that serious data science uses.

# Note 05: What an embedding is (the heart of the project)

Froth project · learning Machine Learning by building it

## The problem: the computer does not understand words

You, as an expert, know these two phrases talk about the same thing:

- "*fine particle flotation*"
- "*recovery of small mineral grains by froth*"

But they share almost no words. If we searched for "matching words", the system would say they have nothing to do with each other. And the other way round: "river bank" and "bank account" share the word "bank" and are not alike at all.

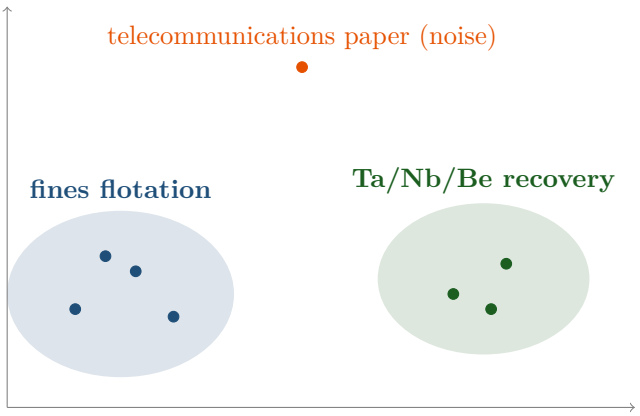
**Conclusion:** counting equal words does not capture *meaning*. We need something else.

## The solution: turn the text into a point on a map

**Concept: embedding (meaning vector)**

An **embedding** is a text turned into a list of numbers, its *coordinates*, so that it can be placed as a **point** on a "map of meaning". The list of numbers is also called a **vector**. The magic property: the map is built so that **texts with similar meaning land close together**, and different ones far apart. Two papers studying the same thing fall together even if they use different words.

Picture the meaning map of your papers (here in 2D so it can be drawn):




Each point is a paper. The flotation ones group on the left; the by-product ones on the right; and the off-topic paper that slipped in (remember the telecommunications one?) sits *alone, far from everything*, which is why the system will later be able to set it aside automatically.

## Why 768 coordinates and not 2?

**Concept: dimensions**

On a real map you use **2** coordinates: latitude and longitude. But meaning is much richer than a position on a plane, so more "room" is needed. The model we will use (SPECTER) describes each text with **~768 coordinates**.
You cannot draw that (nobody sees in 768 dimensions), but it does not matter: the **mathematics** can still measure how close two points are even with 768 coordinates. In Note 06 we will see how ("cosine similarity"). In Phase 3 we will flatten those 768 down to 2 precisely so we can draw the real map.

Each of those 768 numbers captures some abstract "aspect" of the text. They are not interpretable one by one (number 348 does not mean "how much it talks about lithium"); what matters is the *whole pattern* of the 768 together.

## Where do the numbers come from? A pretrained model

**Concept: pretrained model**

The 768 numbers are produced by **SPECTER**: a *neural network* that the AllenAI organisation already trained by reading millions of scientific papers (and who cites whom). It learned to place similar texts close together.
The key point: we **train nothing**. We download the finished model and use it, like someone using a calculator without manufacturing it. That is a *pretrained model*, and it is the fastest and most powerful way to start in text ML.

**Watch out / common mistake**

Why SPECTER and not a generic model? Because it is trained **on papers**. A general-purpose model understands ordinary text; SPECTER understands academic language (methods, results, scientific jargon) far better for our case. A specialised tool for a specialised job.

## What we will do with this (what comes next)

1. Install the libraries that load SPECTER (next step).
2. Turn your 126 abstracts into 126 vectors (a $126 \times 768$ matrix).
3. Measure likeness with cosine similarity (Note 06).
4. Build the **semantic search engine**: you type a phrase → it gives you the 5 papers closest in meaning. (The "wow" moment.)

**In one sentence**

An embedding turns each text into a point (768 coordinates) inside a "map of meaning" where alike lands close; those numbers come from SPECTER, a model already trained on papers that we merely reuse.

# Note 06: Loading SPECTER and your first vector (using vs. building a model)

Froth project · learning Machine Learning by building it

## What we did

In Note 05 we saw the *idea* of the embedding. Here we touch it for real: we take an actual paper from your table and turn it into its vector.

1. We load a paper (title + abstract) from `2_Datos/processed/papers.parquet`.
2. We load the model in one line: `SentenceTransformer("allenai-specter")`.
3. We convert it: `model.encode(text)` → a vector of **768 numbers**.

The test paper was *"Electrostatically Controlled Enrichment of Lepidolite via Flotation"* and its vector started with `[-0.59, -0.16, 1.34, ...]`. That paper is now a point on the map of meaning.

**Concept: pretrained model, in practice**

The first time you call `SentenceTransformer("allenai-specter")`, the library **downloads** the model (~440 MB) from the Hugging Face repository and stores it in a *cache* on your disk. Later times it loads from there, instantly, with no internet. You train nothing: you use weights other people already computed.

**Watch out / common mistake**

**Title + abstract, not the abstract alone.** SPECTER was trained to receive the title glued to the abstract, because the title carries strong signal about the topic. That is why the code does `text = title + ". " + abstract` before vectorising. A small detail that improves the results.

**Watch out / common mistake**

**The *warnings* that came up are harmless.** One mentioned "symlinks" (a disk optimisation on Windows) and the other "HF_TOKEN" (an optional key for faster downloads). Neither prevents anything: the model loaded and worked. Learn to tell a *warning* (a notice, carry on) from an *error* (it stops).

## The key question: are we building the model?

**No.** And it pays to be crystal clear about the difference between two things that sound alike:

| | **USING a model** | **TRAINING a model** |
|---|---|---|
| What do you do? | You download a finished one and feed it text | You teach it from scratch with data; you adjust its "weights" |
| What does it cost? | One line of code, seconds | Weeks, lots of data, graphics cards (GPUs) |
| In Froth? | **Phases 2 to 5** (what we are doing now) | **Phase 6**, optional and advanced |

**Concept: why reusing is the right call (for now)**

To draw the bubbles and the map, generic SPECTER, trained on millions of papers from every field, is more than enough. Training your own embedding model from scratch would be reinventing the wheel: hugely expensive and with no clear gain at this stage. Project rule: *recycle without shame*. What is original about your work is in **how you combine** the pieces and the problem you solve, not in re-manufacturing the engine.

## So when do we actually touch "the model"?

In **Phase 6** (the last one, optional). There the experiment with thesis potential is to *fine-tune* a model with papers from your own domain (minerals, flotation, metallurgy) and measure whether it produces **better** clusters than generic SPECTER. That really is "building something of your own", and by then you will have learned PyTorch and how a model works inside. It is dessert, not the main course.

**In one sentence**

The vector model (SPECTER) is already built: we only download and use it, we do not train it. Using $\neq$ training; training your own is Phase 6, optional. For now, relax: the engine is already made.

# Note 07: Vectorising the 126 papers: the embedding matrix

Froth project · learning Machine Learning by building it

## From one to all

In Note 06 we turned *one* paper into its vector. Here we convert all **126 at once** and **save** the result to disk. That is the key difference: before we were only looking; now we produce data that the next phases will reuse.

**Concept: matrix (N rows × M columns)**

A vector is a row of numbers. Stack many vectors and you get a **matrix**: a table of numbers. Ours has shape `(126, 768)`:

- **126 rows**: one per paper.
- **768 columns**: the coordinates of each paper.

*Row i* of the matrix is the vector of *paper i* in the table: same order. That alignment is sacred, it is how we know which vector belongs to which paper.

## Batch processing: why it is faster

The code does not call the model 126 times (once per paper). It hands over the **whole list** of texts and the model processes them in *batches* (you saw the `Batches: 4/4` bar).

**Concept: batch**

Processing in batches means giving many examples together so the model makes better use of the processor (it does the arithmetic in parallel). One by one would work, but it would be slower. Our 126 papers took ~45 seconds on CPU; with a graphics card it would be faster still, but at this size there is no need.

## Saving to disk: the `.npy` file

Vectorising costs seconds; we do not want to repeat it every time. So we save the matrix in `2_Datos/embeddings/vectors.npy`.

**Concept: the .npy format (NumPy)**

`.npy` is NumPy's own format for storing arrays and matrices of numbers as they are, without converting to text. It is written with `np.save()` and read back instantly with `np.load()`. Our 126×768 matrix weighs barely ~378 KB. The next phases (reduce to 2D, cluster) will read this file instead of recomputing.

**Watch out / common mistake**

**Golden rule of the pipeline:** each phase leaves its result saved and the next one reads it. Papers → `papers.parquet`; vectors → `vectors.npy`. If something breaks further down the line, you do not have to download papers again or recompute embeddings: you pick up from the last saved file.

## What got built

- The `embed_papers(df)` function in `froth/embed.py` (no longer empty).
- The file `2_Datos/embeddings/vectors.npy` with the 126×768 matrix.
- The `save_embeddings()` and `load_embeddings()` helpers for the phases to come.

**In one sentence**

We stacked the 126 vectors into a matrix (126×768), computed it in batches (fast) and saved it in `vectors.npy` so it is not recomputed; row i is always paper i.

# Note 08: Cosine similarity: measuring how alike two papers are

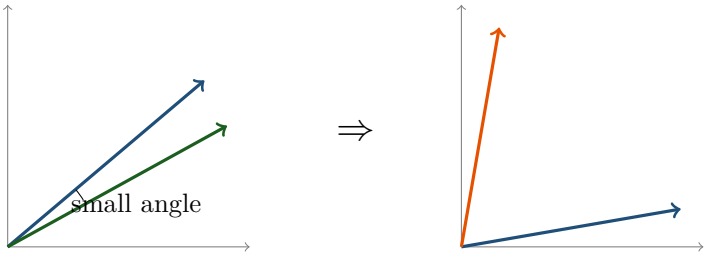
Froth project · learning Machine Learning by building it

## The problem

You already have 126 points (vectors) on the map of meaning. Natural question: given two papers, how does the computer put a number on "how alike they are"? With **cosine similarity**.

## The idea: the angle between two arrows

Each vector is an **arrow** leaving the origin and pointing at the paper's position. To compare two papers we look at the **angle** between their arrows:




**alike**
cosine ≈ 1

**different**
cosine ≈ 0

**Concept: cosine similarity**

It is the **cosine of the angle** between two vectors. It gives an easy number to read:

- Arrows nearly aligned (small angle) → cosine **close to 1** → same topic.
- Perpendicular arrows (90°) → cosine **close to 0** → unrelated.

For text embeddings the value usually falls between 0 and 1.

**Watch out / common mistake**

Why the *angle* and not the *distance* between the tips? Because what matters is **which way** the meaning points, not how long the arrow is. Two papers on the same topic, one with a long abstract and one with a short one, will have arrows of different "size" but pointing the same way: the angle recognises them as equal; straight-line distance would be fooled by the size.

## The test on your papers (real numbers)

We took the paper *"Enrichment of Lepidolite via Flotation"* and computed its cosine against the other 125. This came out:

| Cosine | Paper | Does it make sense? |
|---|---|---|
| 0.942 | Selective Inhibition ... Flotation of Lepidolite | yes: same topic |
| 0.941 | Froth Flotation of Lepidolite, A Review | yes: same topic |
| 0.932 | Flotation Collectors for Lepidolite Mineral | yes: same topic |
| 0.511 | ...Radio Resources for Multicell Mobile | no: telecommunications |
| 0.507 | Adrienne Rich and the Poetry... | no: poetry |
| 0.495 | Theodore Beza... Peace in France 1572 | no: history |

The lepidolite flotation papers came out with a cosine of ∼0.94 (nearly identical in topic), and the off-topic noise that slipped in during Phase 1 sat at ∼0.50 (far away). The system sorted them correctly **knowing nothing about mining**: purely from the meaning of the abstracts.

Concept: the tool: scikit-learn's `cosine_similarity`

We do not compute the angle by hand. `sklearn.metrics.pairwise.cosine_similarity` takes vectors and returns the cosines. We passed it one paper against the matrix of 126 and got all 126 similarities in one go.

## What this is for

This number is the building block of the **semantic search engine** (next step): to find the papers closest to a phrase, we turn the phrase into a vector and keep the ones with the *highest* cosine. And in Phase 3, that same idea of "who is near whom" is what forms the subtopic bubbles.

In one sentence

Cosine similarity measures the angle between two vectors: $\approx 1$ same topic, $\approx 0$ unrelated. On your papers, flotation vs flotation gave 0.94 and flotation vs noise 0.50: it works.

# Note 09: The semantic search engine (putting it all together)

Froth project · learning Machine Learning by building it

## What we built

A tool that takes a **phrase in natural language** and returns the papers closest *in meaning*, not by shared words. It is the `most_similar()` function in `froth/embed.py`, and it brings together everything learned in Phase 2.

**Concept: semantic search vs. keyword search**

Classic search (Ctrl+F, basic Google) matches **words**: if you search "grain size" it does not find a text saying "coarse and fine particles", even though they mean the same thing. **Semantic** search matches **meaning**: it turns your phrase into a vector and looks for nearby vectors, so it does relate them.

## How it works inside (3 steps)

1. **Vectorise the phrase.** Your phrase goes through the *same* model (SPECTER) with `model.encode([query])`. Key point: the phrase and the papers must live on the same map, or comparing them would make no sense.
2. **Measure against everything.** With `cosine_similarity` we compute the cosine of your phrase against all 126 vectors at once.
3. **Keep the best.** We sort from highest to lowest cosine and return the first `k` (5 by default), as a `score, title, year` table.

**Watch out / common mistake**

The same model for both things. If you vectorised the papers with SPECTER and the phrase with a different model, the two "maps" would not line up and the results would be rubbish. Embedding consistency = golden rule.

## The test (real results)

Search: *"how does grain size affect the recovery of minerals by froth flotation"*

| Score | Paper |
|---|---|
| 0.873 | Role of Bubble Size in Flotation of Coarse and Fine Particles |
| 0.856 | Effect of Clay Slime on the Froth Stability and Flotation Performance |
| 0.855 | Spodumene Flotation Mechanism |

We typed "grain size" and it found papers about "coarse and fine particles" and "ultrafine": **it understood the meaning, not the words**.

Search: *"extracting beryllium and tantalum as valuable by-products"*

| Score | Paper |
|---|---|
| 0.812 | Niobium and tantalum |
| 0.740 | Concentration of Tantalum and Niobium |

It brought back exactly the Ta/Nb/Be by-product papers from your thesis.

**Concept: a score between 0 and 1**

The `score` is the cosine similarity (Note 08). Here it hovers around 0.7 to 0.9: high, and it makes sense. Careful: "free phrase vs. paper" scores tend to be somewhat lower than "paper vs. paper" (0.94 earlier), because a short phrase is less rich than a whole abstract. What matters is the *ordering*, not the absolute number.

## What this means for the project

You now have a genuinely useful tool: interrogating your corpus in natural language. And more importantly: the machinery (vectorise + cosine + sort) is **the same** one that will form the subtopic bubbles in Phase 3. Mastering this is mastering the core of Froth.

**In one sentence**

The semantic search engine turns your phrase into a vector, compares it by cosine against the 126 papers and returns the closest in meaning. Tested: "grain size" found "fine/coarse particles". Froth's engine is already beating.

# Note 10: Jupyter notebooks: your interactive search engine

Froth project · learning Machine Learning by building it

## What a notebook is and why it matters

**Concept: Jupyter notebook**

A **notebook** (a `.ipynb` file) is a document made of **cells** you can run one at a time. There are two kinds of cell: **code** (Python that runs) and **text** (explanations in Markdown). You run a cell with **Shift + Enter**.

The point of it: you try something, you see the result right underneath, you change something and run again, without restarting the whole program. That is why it is **the** exploration tool in data science. On top of that, a well-made notebook is an excellent portfolio piece: it tells a story (text + code + results) that anyone can follow.

## Your notebook: `4_Notebooks/01_buscador_semantico.ipynb`

It wraps the `most_similar()` function in a comfortable interface:

1. A **setup** cell loads the 126 papers and their vectors.
2. A **query** cell where you change the `my_query` variable to whatever phrase you want, and run.
3. The result shows up as a **table** (score, title, year, citations), because `most_similar` returns a DataFrame and Jupyter renders those nicely.

**Try it yourself (hands on the keyboard)**

Open it yourself and validate as the expert. In PowerShell, inside the project folder:

1. `cd "C:\Users\you\Documents\Froth"`
2. `.venv\Scripts\python.exe -m jupyter notebook "4_Notebooks\01_buscador_semantico.ipynb`

It opens in your browser. Run the cells with Shift+Enter, change `my_query` and try phrases from your field. **You are the judge** of whether the papers that come out are the right ones: that is the validation that closes Phase 2.

**Watch out / common mistake**

The notebook starts with `sys.path.insert(...)` pointing at the project folder. That is so Python finds the `froth` package even though the notebook lives in the `4_Notebooks/` subfolder. Without that line, `from froth import embed` would fail.

## Notebook vs. code (`.py`): when to use which?

| **Notebook (`.ipynb`)** | **Module (`.py`)** |
|---|---|
| Exploring, trying ideas, teaching, presenting results. | Stable code that gets reused (the "engine", like `embed.py`). |
| Read top to bottom like a story. | Imported from other files. |

A healthy rule: explore in the notebook; when something works and you are going to reuse it, move it to a `.py` in the package. Your search engine already lives in `embed.py`; the notebook only *uses* it.

**In one sentence**

A notebook is a document of runnable cells (Shift+Enter), ideal for exploring and presenting. Yours wraps `most_similar` so you can type queries and see the papers in a table: use it to validate Phase 2 with your expert judgement.

# Note 11: Your first web app with Streamlit

Froth project · learning Machine Learning by building it

## What we built

A **web page that runs on your own PC**: a search box where you type a phrase and see the most relevant papers in a table. It is the `app.py` file at the project root, and inside it uses the *same* `most_similar()` from Phase 2. No new engine: just a nice face on top.

**Concept: Streamlit**

**Streamlit** is a library that turns a Python script into an interactive website without you having to know HTML, CSS or JavaScript. You write `st.title(...)`, `st.text_input(...)`, `st.dataframe(...)` and Streamlit draws the page. Every time the user changes something (types, moves the slider), Streamlit **re-runs the script** from top to bottom and refreshes the view.

## How to start it

**Try it yourself (hands on the keyboard)**

From the project folder, in PowerShell:

1. `cd "C:\Users\you\Documents\Froth"`
2. `.venv\Scripts\python.exe -m streamlit run app.py`

A tab opens by itself at `http://localhost:8501`. Type a query, move the results slider and look at the table. To shut it down: **Ctrl + C** in PowerShell.

**Watch out / common mistake**

"localhost" means "this very machine". The site is NOT on the internet: it lives and runs on your PC. That is why you pay for no domain and no hosting. For others to use it: either they download the repo and run it the same way, or you upload it for free to Streamlit Community Cloud / Hugging Face Spaces (see below).

## Two tricks in the code worth understanding

**Concept: cache: do not reload the heavy things on every click**

Since Streamlit re-runs the script on every interaction, loading the 126 papers and their vectors each time would be slow. The `@st.cache_resource` decorator on the `load_data()` function makes them load **once** and be reused. The SPECTER model is cached the same way (a global variable in `embed.py`).

On top of that, `app.py` imports `from froth import config, embed`: the website is only a client of the engine you already have. If tomorrow you improve `most_similar`, the website improves by itself.

## How it gets shared (your portfolio goal)

- **Download and run:** you push the repo to GitHub; anyone does `git clone`, installs `requirements.txt` and runs it on their PC. Free. (GitHub stores the code, it does not run it for the visitor.)
- **Free public link (optional):** Streamlit Community Cloud or Hugging Face Spaces host the app for free and give you a public URL. No paid domain.

## What comes next (the vision)

This app is the **Thesis module**: input = a title or phrase. Later comes the **Task/Research module**: you attach instruction PDFs and write what you are being asked for, and the same engine returns the relevant papers. And in Phase 3, on top of the list, the subtopic **bubble map** will appear right here.

In one sentence

Streamlit turns your script into a local website (localhost) without knowing HTML. `app.py` wraps `most_similar` in a search box; you start it with `streamlit run app.py`, share it via GitHub or free hosting, and it is the base of the two future modules.

# Note 12: Dimensionality reduction: from 768 to 2 without losing the neighbourhood

Froth project · learning Machine Learning by building it

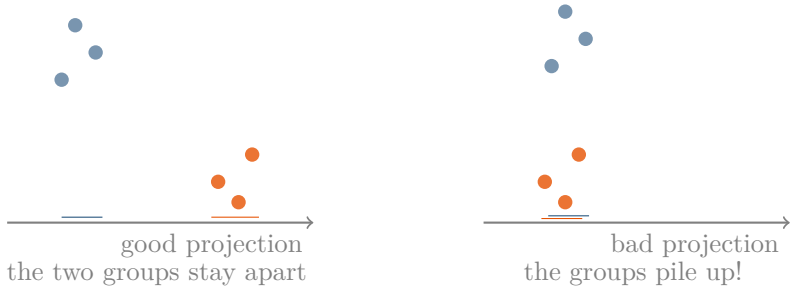
## The problem

Each paper is a point in **768 dimensions** (Notes 05 to 07). Your screen has **2**. To draw the map you have to "flatten", but a bad flattening piles up points that have nothing to do with each other, and then the map lies. The question of this note: *how do you flatten while preserving who is near whom?*

## The classic intuition: PCA and shadows

**Concept: PCA (principal component analysis)**

Think of the **shadow** of your hand on the wall: a 3D object projected into 2D. Orient the hand well and the shadow keeps the shape (you can see the fingers); orient it badly and everything overlaps. **PCA looks for the "light angle" that loses the least information**: the directions in which the data varies most. It is the classic technique, fast and widely used.




The limitation: PCA only makes *straight* (linear) projections. If the data forms curved or tangled clouds, as tends to happen with text, even the best angle piles things on top of each other.

## The modern one: UMAP and moving neighbourhoods house by house

**Concept: UMAP**

UMAP does not project shadows. It works in two stages:

1. In 768D, it notes for each paper **who its nearest neighbours are**.
2. It places the points in 2D, bending and stretching whatever it needs to, with a single obsession: **that every point keeps its neighbours beside it**.

Like moving a city onto a smaller map while guaranteeing that each house stays next to the same houses as before, even if the long distances change. That is why UMAP separates clouds that PCA piles up: it is not restricted to straight projections.

## The small print: every 2D map lies a little

**Watch out / common mistake**

Flattening 768 dimensions into 2 **always** loses something. On a UMAP map:

- **Trust**: who is stuck to whom; the clouds and groups that form.
- **Do NOT trust**: the long distances ("this cluster is twice as far as that one") nor the apparent size of the clouds.
- The X and Y axes **mean nothing** (they are not "more flotation" or "more lithium"); they are just the result of the arrangement. That is why UMAP maps are published without numbers on the axes.

Knowing this puts you ahead of a lot of people who over-read these maps.

## How we will use it (what comes next)

1. Install `umap-learn` (if Python 3.14 allows it; there is a plan B with scikit-learn's PCA/t-SNE, which changes the part, not the concept).
2. Run UMAP on the $126 \times 768$ matrix $\rightarrow$ a table of 126 points (x, y).
3. Draw the first scatter plot: your first glimpse of the map (Note 13).
4. Then group the clouds into subtopics with HDBSCAN (Note 14).

**In one sentence**

PCA flattens by looking for the best "shadow angle" (fast but straight); UMAP rearranges while preserving neighbourhoods (ideal for text). On the resulting map: trust the neighbours and the clouds, distrust long distances, and the axes mean nothing.

# Note 13: UMAP in action: the first map of your 126 papers

Froth project · learning Machine Learning by building it

## What we did

We ran UMAP for real on the $126 \times 768$ matrix and out came a $126 \times 2$ table: one (x, y) coordinate per paper. We saved them in `2_Datos/embeddings/umap_2d.npy` and drew the first scatter plot (it is in `3_Resultados/umap_first_look.png`).

**The important part of the result:** without being told anything about mining, *structures* are already visible: a big cloud on the left, an arm at the top right, a diagonal strip at the bottom with a tight clump in the corner. Those are (probably) your subtopics waiting to be named. The next step (HDBSCAN) will delimit them formally.

## The three decisions in the code (and why)

**Concept: metric="cosine", measuring with the same yardstick**

UMAP needs to decide who each point's "neighbours" are, and for that it measures distances. We told it to measure them with **cosine similarity**, exactly the same yardstick as Phase 2 (Note 08). If Phase 2 compares papers by cosine and UMAP used a different measure, the map would tell a different story from the search engine. Consistency above all.

**Concept: random_state=42, reproducibility**

UMAP has internal randomness (where the arrangement starts from). With no fixed seed, each run would give a *slightly* different map: same groups, different positions. Fixing `random_state=42` means anyone running your code gets **the same map**. In science (and in your thesis) that is called *reproducibility* and it is sacred. (The 42 is a programmers' in-joke; any fixed number works.)

**Concept: n_components=2, how many output dimensions**

We asked for 2 dimensions because the goal is to *draw*. You can reduce to 5 or 10 (sometimes useful for clustering); we will explore that if it is needed.

**Watch out / common mistake**

In the scatter plot we removed the numbers on the axes **on purpose**. After Note 12 you already know why: in UMAP the axes mean nothing. If one day you see a UMAP map with numbered, interpreted axes ("the higher the X, the more of such and such"), distrust the author.

## How to read your first map

- **Dense clouds** = groups of papers talking about the same thing (future clusters).

- **Loose points / bridges** = papers that mix topics or do not fit (candidates for "noise" in HDBSCAN).
- **Overall shape** (arm, strip, clump): it suggests subtopics with different cohesion. A tight clump is a very specific topic; a broad cloud, a general one.

In one sentence

UMAP turned 768 dimensions into a reproducible 2D map (fixed seed) measuring neighbours by cosine (the same yardstick as always), and the clouds appeared on their own: they are the subtopics HDBSCAN will delimit in the next step.

# Note 14: Clustering: letting the algorithm find the groups (HDBSCAN)

Froth project · learning Machine Learning by building it

## The problem

Your 2D map (Note 13) shows clouds to the naked eye, but "to the naked eye" is not a method: we need an algorithm to delimit the groups objectively and repeatably. That is **unsupervised clustering**: grouping without anyone saying what each group is or how many there are.

## The classic intuition: K-means (and its two traps)

Concept: K-means

The most famous method. You tell it "I want **K** groups" and it splits the points into K compact groups. Simple and fast, but with two traps for our case:

1. **You have to guess K.** How many subtopics does your topic have? That is exactly what we do NOT know (it is what we want to discover).
2. **It forces EVERY point into some group.** The poetry paper would end up assigned to a flotation cluster, contaminating it.

## What we use: HDBSCAN, density-based clustering

Concept: HDBSCAN

Instead of splitting, HDBSCAN looks for **dense zones**: regions where many points sit tightly together. Each dense zone = one cluster. Consequences:

- **It decides by itself how many groups there are** (we do not guess K).
- Whatever belongs to no dense zone is left as **noise** (label $-1$), instead of contaminating a group.
- Its main parameter is `min_cluster_size`: the minimum number of points needed for something to count as a group (in Froth: `MIN_CLUSTER_SIZE = 8`).

## What happened with YOUR papers (the real story)

With `min_cluster_size=8`, HDBSCAN found **3 clusters + 9 noise papers** (and with 5, almost identical: a *stable* result, a good sign):

| Group | Size | What it turned out to be (Note 15) |
|---|---|---|
| Cluster 0 | 49 | Geology of pegmatites and rare-metal granites |
| Cluster 2 | 43 | Lepidolite flotation (the heart of the thesis) |
| Cluster 1 | 25 | "Drawer of other people's things": lithium recycling/economy + the off-topic ones |
| Noise | 9 | Generalist bridges and oddities |

Watch out / common mistake

**Two surprises that teach a lot:**

1. **"Noise" is not always rubbish.** Two entirely on-topic classics landed in the noise (for example *The Effect of Bubble Size on Fine Particle Flotation*, 527 citations). Why? They are *generalist* papers that speak to several subtopics at once: they live "between" the clouds, without fully belonging to any.
2. **The off-topic ones were not noise: they were quarantined together.** The telecommunications paper, the poetry one and the history one all three fell inside cluster 1. "Not being on topic" is also something in common, and it forms its own cloud far from the mining ones.

In one sentence

K-means demands that you guess K and forces everyone in; HDBSCAN groups by density, decides by itself how many groups there are and sets the noise aside. On your data: 3 stable subtopics + 9 noise papers, with the off-topic ones quarantined inside their own cluster.

# Note 15: Naming the bubbles: c-TF-IDF

Froth project · learning Machine Learning by building it

## The problem

HDBSCAN gives you groups called “0”, “1”, “2”. For the map we need names with meaning. How do you name a group of 49 papers *automatically*?

## Why the most frequent words do NOT work

If you take the most repeated words in each cluster, the same ones come out in ALL of them: “mineral”, “flotation”, “study”, “results”. Frequent ≠ characteristic. What distinguishes a group are the words it uses a lot **and the others use little**.

**Concept: TF-IDF and its class-based version (c-TF-IDF)**

**TF-IDF** scores each word by combining two factors: how frequent it is in a document (TF) and how *rare* it is in the rest (IDF). Frequent here + rare elsewhere = high score = characteristic.
The **c-TF-IDF** (class-based) variant applies this to clusters: all the abstracts of each cluster are glued together as if they were **a single mega-document**, and the mega-documents are compared against each other. The top words of each mega-document are the cluster’s label. It is the same trick BERTopic uses inside.

## What came out with YOUR clusters

| Group | Keywords (top c-TF-IDF) | Reading |
|---|---|---|
| Cluster 0 (49) | melt, crystallization, Nb, Zr, Hf | Geology/petrology of pegmatites and rare-metal granites |
| Cluster 2 (43) | flotation, collector, froth, lepidolite | Lepidolite flotation: collectors and froths |
| Cluster 1 (25) | commodity, old scrap, USGS, women, men | Lithium economy/recycling + the off-topic papers (quarantine) |

The map makes complete mineralogical sense: on one side the geology of the deposit, on the other the processing (flotation), and separately “the rest of the world”.

**Watch out / common mistake**

**A data quality finding:** among the flotation cluster’s keywords, some *French* words slipped in (“du”, “récupération”). Cause: the corpus contains French abstracts (Beauvoir is in France!) and our *stopword* list (empty words that get ignored: the, of, and...) only covers English. It breaks nothing, but it dirties the labels. Possible fixes: multilingual stopwords, or detecting the language during the Phase 1 cleanup. Lesson: real data always brings one more surprise.

**Concept: the algorithm proposes, the expert validates**

Automatic labels are a *draft*: "melt-crystallization-Nb" is useful, but you as the expert will refine it into "Petrogenesis of rare-metal granites". That division of labour (machine proposes, human curates) is exactly how these tools are used in serious research.

**In one sentence**

c-TF-IDF names each cluster with its characteristic words (frequent inside, rare outside), by gluing each cluster into a mega-document. Your 3 subtopics: pegmatite geology, lepidolite flotation, and economy/recycling (with the off-topic ones in quarantine).

# Note 16: The interactive bubble map (Plotly) and the slider trick

Froth project · learning Machine Learning by building it

## What we built

The star deliverable of Phase 3: the **interactive bubble map** of your 126 papers. It exists in two forms, fed by the same code:

1. `3_Resultados/topic_map.html`: a self-contained file. Double click and it opens in any browser, no server needed. Shareable.
2. The **Topic map** tab of your web app (`app.py`), with live controls.

What you see: one dot per paper, one colour per subtopic (geology blue, flotation green, economy/other orange, noise grey), size = citations, each subtopic's label floating over its cloud, and on hover: title, authors, year, citations.

**Concept: Plotly**

A library for **interactive** charts: instead of a fixed image (matplotlib), it produces HTML + JavaScript with zoom, pan, tooltips and a clickable legend "for free". That is why the map can be saved as a file and opened anywhere: the browser does the work.

**Concept: one figure, two faces (architecture)**

The `bubble_figure()` function in `froth/visualize.py` builds the figure, and TWO clients reuse it: `plot_bubbles()` saves it as HTML, and `app.py` shows it in Streamlit. If tomorrow you improve the map, it improves in both places at once. General principle: *you do not duplicate the engine; you wrap it.*

## The granularity slider trick

The web app has a "min papers per subtopic" slider that re-groups the map **instantly**. How can it be so fast, if the pipeline takes time?

**Concept: separating the slow from the fast**

The pipeline has two very different parts:

- **Slow**: UMAP (arranging 126 points down from 768D). Computed ONCE and cached.
- **Fast**: HDBSCAN over the already-arranged 2D points (milliseconds) + relabelling.

The key: granularity **does not affect UMAP**, only HDBSCAN. So moving the slider only repeats the cheap part. Identifying what can be cached and what must be recomputed is a core data engineering skill, and you have just used it.

Watch out / common mistake

Granularity is a **reading decision**, not a truth: with a low value you will see many small subtopics (more detail, more fragmentation); with a high value, a few big ones (more synthesis). There is no "correct number of clusters", there is the one that best tells the story for your purpose. That is why it is your slider and not a hidden constant.

## Where we stand against the competition

With this map we reach visual and interactive **parity** (hover, zoom, filters, size by citations) with Connected Papers, Litmaps or VOSviewer. And we already have differentiators they do not: a map by *content* (not citations), auto-named subtopics, live granularity, separable noise, integrated semantic search, and all of it open source. The star differentiators (gaps + review draft + task/PDF module) arrive in Phases 4 and 5. The full table lives in `CLAUDE.md`.

In one sentence

Plotly turns the topic map into an interactive map with two faces (shareable HTML and an app tab) from a single function; the granularity slider is instant because it only repeats the cheap part (2D HDBSCAN), not the expensive one (UMAP). Parity with the competition: achieved; the big differentiators come in Phases 4 and 5.

# Note 17: The relevance filter: the system cleans itself

Froth project · learning Machine Learning by building it

## The problem YOU spotted

While validating the map (step 3.7) you saw a cluster labelled *"women, men, commodity, masculinities"*. Diagnosis: the papers that infiltrated in Phase 1 (poetry, telecommunications, history, from acronym collisions in the searches) dominated the keywords of a cluster that also contained legitimate lithium economy papers. The root cause was in the *pull*, not in the map.

## The elegant fix: measure every paper against the TOPIC

**Concept: relevance score**

We already have all the pieces: each paper has its vector, and the **title of your thesis** can have one too. So: *a paper's relevance* = cosine similarity between its vector and the TOPIC's vector. A poetry paper lands far from the title of a flotation thesis → low score → out. The system uses **its own embeddings to clean itself**: no manual rules, no blacklists.

**Concept: the threshold is dictated by the data, not by intuition**

Where to cut? We looked at the real distribution of the 126 scores:

| | |
|---|---|
| Infiltrators (poetry, telecom, history...) | 0.495 to 0.543 |
| **(natural gap)** | |
| Everything legitimate (including Li economy) | 0.587 to 0.90 |

There is a **gap** between 0.543 and 0.587: the threshold `RELEVANCE_THRESHOLD = 0.55` falls in the middle and separates them perfectly. That is how thresholds are chosen seriously: by looking at where the data is already separated, not by inventing a nice-looking number.

## Where it lives in the code

- `embed.py` → `relevance_to_topic()`: computes the scores and saves them as a `relevance` column in `papers.parquet`.
- `config.py` → `RELEVANCE_THRESHOLD = 0.55` (with the reasoning in a comment).
- `cluster.py` → filters BEFORE UMAP: the rubbish does not even take part in the arrangement, so the whole map improves.

## What changed on the map (a real data science iteration)

- The embarrassing labels disappeared; the economy cluster is now honest (*usgs, old scrap* = recycling/commodities).
- A **4th cluster** emerged that had been buried: literature **in French** about Nb-Ta-Sn recovery, probably the French school studying Beauvoir. A valuable finding, and a new lesson: *language also groups* (SPECTER puts the French papers together). Future refinement: translate abstracts, or accept that cluster as a valid subtopic?

- The noise went up (9 → 21): as the map rearranged, more papers ended up between clouds. That is what your granularity slider is for.

**Watch out / common mistake**

This is what the real data science cycle looks like: **build → look → spot something odd → diagnose → fix → discover something new**. Your expert eye spotted the problem; the embeddings provided the solution; and the fix uncovered a finding (the French cluster) that nobody was looking for. Every iteration teaches.

**In one sentence**

Relevance = cosine(paper, TOPIC); threshold 0.55 chosen from the natural gap in the data; filtered before UMAP. Result: a clean map, honest labels and a surprise French cluster about Beauvoir. The system cleans itself with its own tools.

# Note 18: Git and GitHub: save points and cloud backup

Froth project · learning Machine Learning by building it

## What Git is

**Concept: Git = a save-point machine**

Like in a video game: every time you reach a state worth keeping, you make a **commit** ("save the game"): Git photographs ALL the project's files at that instant and keeps the photo forever, with a date, an author and a message. If you break something tomorrow, you can go back to any earlier photo. The history lives in the hidden `.git` folder. Every programmer on the planet uses it, daily.

Minimum vocabulary:

- **repo(sitory)**: the project watched over by Git.
- `git add .`: put files "in the tray" for the next photo.
- `git commit -m "message"`: take the photo.
- `git status`: see what is in the tray and what changed.
- **branch**: a line of history. The main one is called `main`.
- **remote / origin**: the nickname of the copy in the cloud.
- `git push`: push your local photos to the cloud.

**Concept: .gitignore = what NEVER enters the photos**

A text file with the list of the forbidden: secrets (`.openalex_key`), the bubble (`.venv/`), regenerable data (`2_Datos/`), downloaded tools (`tools/`). That way the repo carries the *code and the knowledge*, not the dead weight nor the keys. We checked with `git ls-files` that no secret got in: zero.

## What GitHub is (and what it is NOT)

**GitHub** is the cloud where you back up Git repos, and also your **public portfolio**: anyone can see your code, clone it (`git clone`) and run it on their PC. GitHub *stores* the code; it does not *run* it for the visitor.

Your repo: `github.com/kmortizva-data/froth`, **private** for now (GitHub shows strangers a 404 so as not to reveal even that it exists). When it is polished: Settings → Change visibility → Public, and the portfolio is born.

## What happened your first time (stumbles included, they teach)

1. `git init` + `git add .` + `git commit` → first photo: 65 files, zero secrets.
2. **Stumble 1**: the push failed with "repository does not seem to exist". The remote pointed at an example URL whose page had never been created on GitHub. Lesson: *the cloud*

*repo has to be created before you push* (or let GitHub Desktop create it with its **Publish repository** button, which does both things).

3. **Stumble 2**: `git remote remove origin` answered "No such remote", which meant "it was already removed", not a real error. Lesson: read the message; sometimes the "error" is just a notice that there was nothing to do.
4. `git branch -M main`: rename `master` → `main` (the modern standard). Silence from the terminal = success.
5. **Publish repository** in GitHub Desktop → it created the cloud repo and pushed. Verified: `main...origin/main` with no differences = in sync.

## Your routine from now on (the only one you need to memorise)

At the end of every work session with changes worth keeping:

`git add .` → `git status` (review) → `git commit -m "what I did"` → `git push`

(Or in GitHub Desktop: review changes → write a summary → Commit → Push. It is the same thing with buttons.)

**In one sentence**

Git keeps complete photos of the project (commits) and GitHub backs them up in the cloud and displays them as a portfolio; the .gitignore protects secrets and dead weight. Your routine: add → status → commit → push. First photo: 65 files, zero secrets, already in the cloud.

# Note 19: Graphs: nodes, edges, hubs and bridges

Froth project · learning Machine Learning by building it

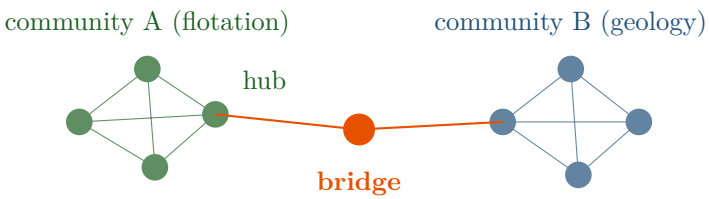
## What a graph is

**Concept: graph = things + relations**

Think of your social network: people = **nodes**, friendships = **edges** (the lines). That structure, things connected by relations, is a **graph**, and it sits behind Google Maps (junctions + streets), LinkedIn (people + contacts) and our mind map (papers + "are related to"). The standard library in Python: `networkx`.

Four ideas we will use:

1. **Weight**: not all relations are equally strong. "They match at 0.94" weighs more than "they match at 0.60", and the line is drawn thicker.
2. **Direction**: similarity is mutual (an edge with no arrow); citations are not, A cites B, not the other way round (an edge *with* an arrow).
3. **Hub**: a node with a great many connections.
4. **Bridge**: a node or edge connecting two communities that barely touch.




## What each one is for in YOUR literature review

**Concept: hubs = foundation; bridges = originality**

- **The hub** is the compulsory reading: the classic everyone cites. It goes in the introduction of your review, it shows you know the ground. But *everybody knows the hubs*: there is no novelty possible there.
- **The bridge** connects two topics that barely speak to each other. If only 2 papers join "flotation" with "deposit geology", that frontier is under-explored → a new contribution fits there. The *gaps* of Phase 5 are, literally, missing bridges.

Short answer: hubs to ground yourself, **bridges to contribute**.

Watch out / common mistake

**Your thesis IS a bridge.** Look at the title: *"Flotation of ... Li bearing mica minerals **and its impact on** the by-products (Ta, Nb, Be,...) recovery..."*. It connects exactly the green cluster (flotation) with the blue one (geology / granite by-products). It is not a topic *inside* a community: it is the *frontier* between two. That is why it is a thesis and not an assignment. When the network is built, look for yourself in it.

## Which graph we will build (what comes next)

1. **Similarity network** (4.2): each paper connected to its k most similar neighbours by cosine **in 768D** (the real space, not the 2D map, see Note 12). With a threshold, so it does not come out as a "hairball".
2. **Citation network** (4.3): arrows for who cites whom. It requires re-downloading the papers asking for the `referenced_works` field that we did not request in Phase 1.
3. **Topic mind map** (4.4): one node per subtopic, edges = how related they are. The presentable view.

In one sentence

A graph is nodes + edges (with weight and sometimes direction). Hubs are the classics that ground you; bridges are the frontiers where originality lives, and your own thesis is a bridge between flotation and geology. We will build similarity, citation and topic networks.

# Note 20: The similarity network: K neighbours, threshold, and your real hubs and bridges

Froth project · learning Machine Learning by building it

## How the network was built

Each paper is connected to its **K most similar papers** (cosine similarity computed in the **original 768D space**, not on the 2D map, which we already know distorts), and only connections with a similarity $\geq$ a **threshold** survive. This is the so-called *kNN graph* (k nearest neighbours).

**Concept: the K and threshold knobs: nobody "knows" them, you test them**

You asked: who says 5 neighbours is ideal? Honest answer: **nobody**. It is a knob, like granularity. The serious way to choose it is to try values and look at the resulting structure:

| K | lines | conn./paper | pieces | % across clusters |
|---|---|---|---|---|
| 3 | 281 | 4.7 | 1 | 17% |
| **5** | **457** | **7.6** | **1** | **21%** |
| 8 | 702 | 11.7 | 1 | 24% |
| 15 | 1213 | 20.2 | 1 | 30% |

A high K reveals more crossings between topics (your first neighbours are family; the distant ones cross borders) but with 20 connections per paper the drawing is unreadable (a "hairball"). Chosen: **K=5, threshold 0.75**: readable, with visible bridges and the network in a single piece. Both live in `config.py` (`GRAPH_KNN`, `GRAPH_SIM_THRESHOLD`): changing your mind costs one line.

## Measuring hubs and bridges (no longer by eye)

**Concept: degree and betweenness**

- **Hub** = a node with many connections → measured by **degree** (count its edges).
- **Bridge** = a node the paths between communities run through → measured by **betweenness centrality**: how many times a node lies on the shortest path between two others. A paper with *few* connections can be a great bridge if those few join worlds.

## The results with YOUR papers

**Hubs** (the corpus's compulsory reading):

1. *The Magmatic-Hydrothermal Transition in Lithium Pegmatites* (24 connections)
2. *Electrostatically Controlled Enrichment of Lepidolite via Flotation* (15)
3. *Mineral Chemistry of Two-Mica Granite Rare Metals* (15)

**Bridges** (where novelty lives), with two big findings:

1. Among the top 5 bridges is *"Characterization and Liberation Study of the **Beauvoir Granite**"*: the founding paper of YOUR deposit is a bridge between communities. The network

confirms **with numbers** that your thesis's territory (geology ↔ processing at Beauvoir) is frontier ground: a measurable originality argument for your introduction.

2. 3 of the top 5 bridges had been labelled **noise** by HDBSCAN. The lesson of Note 14 ("noise ≠ rubbish; they are generalists between clouds") held up quantitatively: *yesterday's noise is today's bridges.*

## Where it lives in the code

`froth/graph.py`: `build_similarity_graph()` (the network, with per-node attributes), `find_hubs()` (degree), `find_bridges()` (betweenness), `save_graph()/load_graph()` (GraphML format in `2_Datos/processed/`). Run it with `python -m froth.graph`.

In one sentence

kNN network: each paper with its K=5 most similar (cosine in 768D), threshold 0.75, knobs chosen by testing and stored in config. Degree finds hubs; betweenness finds bridges. Your corpus confirmed it: the Beauvoir paper is a bridge, and the "noise" was hiding the bridges.

# Note 21: The topic mind map (and the discovery of the silos)

Froth project · learning Machine Learning by building it

## Going up a level: from papers to topics

The network of Note 20 has 120 nodes: good for exploring, bad for *presenting*. The **topic mind map** aggregates the information one level up: each **whole subtopic becomes ONE node** (size = number of papers) and the edges summarise the relation between subtopics. It is the executive view: 3 or 4 bubbles that tell the whole field.

**Concept: aggregation: the same network, less resolution**

This trick, grouping nodes and summarising their relations, is called *aggregation* and it is universal: from people → departments, from streets → cities, from papers → subtopics. You lose detail, you gain legibility. That is why we keep *both* networks: the paper one (to explore) and the topic one (to understand and present).

## The edges carry TWO numbers (and that is the point)

Between each pair of subtopics we measure two different things:

1. **Similarity** = the average cosine between the papers of one and the other (in 768D). "How related is what they *study*?"
2. **Cross-citations** = how many times a paper in one subtopic *cites* one in the other. "How much do they *read* each other?"

## The finding: silos (your gap, quantified)

With your data this came out:

| Relation | Similarity | Cross-citations |
|---|---|---|
| Geology ↔ Flotation | 0.688 | **1** |
| Geology ↔ Economy/recycling | 0.662 | 3 |
| Flotation ↔ Economy/recycling | 0.683 | 1 |

**Watch out / common mistake**

Read the first row with a thesis writer's eyes: geology and flotation **study very related things** (0.69 is high) but **barely cite each other**, 1 citation among 90 papers! Of the corpus's 116 internal citations, only 5 cross subtopic borders: the communities are **silos**, connected by meaning, separated by habit.
Your thesis ("flotation of Li micas *and its impact on* the recovery of granite by-products") lives exactly in that silence between silos. This is a *gap* that is quantified and citable in your introduction: Phase 5 will formalise this detection.

## Where it lives in the code

`froth/graph.py` → `build_topic_graph()`: nodes with label / number of papers / citations / centroid, edges with similarity and cross-citations. It prints when you run `python -m froth.graph`; the preview is in `3_Resultados/topic_mindmap.png`.

**In one sentence**

Aggregating the paper network up to topic level gives the executive view (3 or 4 bubbles). Each edge carries similarity (what they study) and citations (what they read), and in your corpus they diverge: closely related subtopics that do not cite each other = silos = your gap, already with numbers.

# Note 22: The final views: physics network and circle packing

Froth project · learning Machine Learning by building it

## Three views, three questions

Your app now has **four tabs**, and each view answers a different question:

| View | Answers... | Position means... |
|---|---|---|
| Topic map (scatter) | What groups are there and how close are they? | Semantic distance (UMAP) |
| Network | Who connects with whom? Hubs? Bridges? | The physics arranges it; the links are what is real |
| Packed bubbles | How is the field divided up? (presenting) | **Nothing**, only membership and size |

Knowing *what position means in each view* saves you from over-interpreting: in the scatter, closeness IS information; in the packing it is only aesthetics.

## The physics network (pyvis)

**Concept: force-directed layout**

The Network tab simulates **physics**: each link is a *spring* pulling the connected papers together, and all the nodes *repel* each other slightly. The system is let go and finds equilibrium by itself: heavily connected communities end up together and compact, and bridges are left stretched in the middle. That is why **dragging a node** makes the whole network react. The "animation" is not decoration, it is the equilibrium being recomputed. The library: `pyvis` (a wrapper around vis.js, a JavaScript engine).

## The circle packing (circlify)

**Concept: circle packing**

The Packed bubbles view places one **big circle per subtopic** and packs one circle per paper inside it (area = citations), **with no overlaps**. Working out where each circle goes so they fit tightly is a classic geometry problem; the `circlify` library solves it and we only draw (with Plotly, to keep the hover). It is the "presentation" view, the one like the EU packaging chart that inspired the project.

**Watch out / common mistake**

In the packing, two neighbouring papers are NOT more alike than two distant ones: position inside the circle is decided by the packing geometry, not by meaning. Membership (which big circle) and size (citations) are the only information. Elegant for communicating; not for analysing.

## Where it all lives

- `froth/visualize.py`: `network_html()` (self-contained pyvis network), `packed_figure()` (packing with circlify + Plotly), and their `save_*`/`plot_*` versions that write HTML into `3_Resultados/`.
- `app.py`: new **Network** and **Packed bubbles** tabs. The network is cached by data version (the same trick as the slider note).
- Shareable files: `topic_map.html`, `paper_network.html`, `packed_bubbles.html`. Double click and they open in any browser.

In one sentence

Three views for three questions: scatter (semantic distances), physics network (connections, hubs, bridges, draggable because it is a system of springs) and circle packing (pure presentation, position without meaning). All in the app and as shareable HTML files.

# Note 23: DOI, open access and the reading list

Froth project · learning Machine Learning by building it

## The new feature: from ranking to reading

The search engine already ranked papers; now it also takes you to **read** them: each result carries two links, **DOI** (the paper's official page) and **open PDF** (the free PDF, when it exists). In your corpus: **85 of 126 papers (67%) have an open access PDF** and 116 of 126 have a DOI.

**Concept: DOI (Digital Object Identifier)**

The **DOI** is a paper's permanent "ID number": `10.1016/j...`. The URL `https://doi.org/[DOI]` always leads to the publisher's official page, even if the journal changes website. It is the correct way to cite and to reach the text; if the paper is paywalled, that is where you use your university access.

**Concept: open access (OA)**

A paper is **open access** when a legally free copy exists: in the journal itself, in repositories (arXiv, HAL...) or on the author's university page. OpenAlex tracks where that copy is and gives us the URL (`oa_url`). We only asked for one more field in the pull's `select` (`open_access`), one line of code, and the pipeline regenerated everything else by itself.

**Watch out / common mistake**

**Linking yes, mass downloading no.** Froth gives you the LINK and you click it: the reading happens on your side, with your permissions. Downloading PDFs en masse from publishers violates their terms of use (a risk foreseen in the plan from day 1). The serious tools (Connected Papers and the like) do exactly this same thing: rank + link.

## Bonus of the day: Streamlit's slow watchman

The app took a long time to start and the log filled up with `torchvision` warnings. Diagnosis: Streamlit's *file watcher* (auto-reload when you edit code) tries to inspect the hundreds of `transformers` modules, one by one. Fix: turn it off in `.streamlit/config.toml` (`fileWatcherType = "none"`). Double lesson: (1) a noisy log is not always a fatal error, you have to read WHAT it says; (2) comfortable development tools sometimes cost performance.

**In one sentence**

Every search result now links to the DOI (official page) and to the open access PDF when one exists (67% of your corpus). We link, we do not mass download. And the app starts faster after turning off Streamlit's file watcher.

# Note 24: What a gap is (and how it is really measured)

Froth project · learning Machine Learning by building it

## Why they matter

Every thesis has to contribute something new, and "new" almost always means **working where others have not worked**. The problem: the phrase "research gap" is used a lot and measured little, it is usually the supervisor's hunch. Froth's differentiator is turning the gap into a **measurable signal over the data**, with evidence you can cite.

## The 4 measurable types of gap

**Concept: 1. The SILO (the most valuable)**

Two communities studying **very similar** things that **do not cite each other**: they work back to back. It is measured by subtracting two matrices we already have: semantic similarity between clusters (high) vs. cross-citations (low). **Your corpus has one**: geology ↔ flotation = similarity 0.69 with **a single cross-citation** (Note 21). Two worlds talking about the same granite without talking to each other. Whoever connects those worlds contributes.

**Concept: 2. The SPARSE ZONE**

Empty regions of the map *between* dense clouds: combinations of topics nobody touched. It is measured with spatial density in the embedding space (how many papers live between the centroid of A and that of B?).

**Concept: 3. The SCARCE BRIDGE**

Between two clusters there are only 1 or 2 bridge papers (Note 20's betweenness concentrated in very few nodes). If crossing that bridge produces value and almost nobody has crossed it, there is room for more travellers.

**Concept: 4. The DORMANT TOPIC**

A subtopic whose papers are all old: knowledge nobody revisits with modern methods. It is measured with the distribution of years per cluster (when was the last paper?).

## The warning: mine or desert

Watch out / common mistake

A hole can be empty for two reasons: because **nobody explored it** (a mine waiting) or because **there is nothing to find** (a desert, sometimes the combination of topics is sterile for good physical or economic reasons). **No algorithm tells one from the other.** That is why Froth *proposes candidates with evidence* and the expert decides. Distrust any tool promising "guaranteed automatic gaps"; ours promises honest candidates with their numbers.

## Your thesis is already the proof of concept

The thesis title (flotation of Li micas **and its impact on** the recovery of by-products from the Beauvoir granite) exploits *exactly* silo #1: it joins the flotation cluster with the geology one. Your supervisor found that gap by eye and by experience; Froth found it with numbers: high similarity, one cross-citation, and the Beauvoir characterisation paper as a lone bridge (Note 20). That double confirmation (expert + data) is the strongest validation the method can have.

## What comes next (5.1)

Implement the four signals in `gaps.py` → `find_gaps()` will return a ranked table: each candidate gap with its type, its evidence (similarity, citations, density, years) and the frontier papers already circling it. Afterwards (5.2), the review draft will cite those gaps with numbers, not with hunches.

In one sentence

A measurable gap has 4 shapes: silos (alike but not citing each other), sparse zones, scarce bridges and dormant topics. The tool proposes candidates with evidence; the expert tells mines from deserts. Your own thesis is the Beauvoir geology-flotation silo, now confirmed with numbers.

# Note 25: From theory to text: find_gaps() and the review draft

Froth project · learning Machine Learning by building it

## What we built (5.1 + 5.2)

The two halves of the differentiator, already working in `froth/gaps.py`:

1. `find_gaps()`: turns the 4 types from Note 24 into measured signals → a ranked table with evidence (`2_Datos/processed/gaps.parquet`).
2. `draft_review()`: writes the scaffolding of the literature review (`3_Resultados/review_draft.md`) with **100% traceable citations**.

They run together: `python -m froth.gaps`.

## The gaps in YOUR corpus (real results)

| Type | Top candidate | Evidence |
|---|---|---|
| Silo | commodities/criticals ↔ flotation | sim 0.69, **1** cross-citation |
| Silo | flotation ↔ geology (*your thesis*) | sim 0.70, 3 cross-citations |
| Scarce bridge | commodities ↔ geology | only 8 bridges despite sim 0.67 |
| Dormant | (none: hot field, 2025-26 papers) | the tool says so honestly |

A close reading of the champion silo: the *criticality / supply chain* literature and the *processing* literature barely cite each other. Connecting them ("my flotation matters for Li/Ta supply security") is direct ammunition for introductions and proposals.

**Watch out / common mistake**

The weakest signal is the **sparse zone**: it measures *absences*, and a void does not explain its reason (mine or desert?), on top of inheriting the distortions of the 2D map (Note 12). That is why it is labelled "experimental" in the code and in the draft. General rule: distrust metrics about what is missing more than metrics about what exists.

## The draft: why a template and not an LLM

**Concept: traceable citations or nothing**

The draft is generated with a **deterministic template**: each section is born from the MEASURED structure (sections = clusters; must-reads = most cited + local hubs; novelties = most recent; holes = gaps with their numbers) and every statement points at real papers with a numbered reference [n] → a final list with author/year/title/DOI. An LLM would write more elegantly... and sooner or later it would *invent* a reference, the capital sin in academia. Correct order: first the trustworthy scaffolding, then (optionally) an LLM that polishes the prose while *verifying* every citation against the corpus.

## Against the competition (a different neighbourhood)

Here the rivals are not the mappers but Elicit/SciSpace/Consensus (an LLM synthesising *without* knowing the corpus structure), Scholarcy (one-to-one summaries) and plain ChatGPT (hallucinated references). **Nobody joins map and text**: our draft says "these two subtopics share 0.69 similarity and a single cross-citation" because it measured it. Honest limit: we work with abstracts (Elicit extracts from full text) → ours is a *structured scaffolding*, not final prose.

In one sentence

find_gaps() produces ranked candidates with evidence (silos at the front: commodities-flotation 0.69 with 1 citation; your thesis reconfirmed at 0.70 with 3); draft_review() turns them into a review scaffolding with 26 traceable references, with no LLM. Nobody else joins map and text.

# Note 26: Task mode from the inside: chunking and averaging (mean pooling)

Froth project · learning Machine Learning by building it

## The problem you cannot see

Task mode accepts whole instruction PDFs. But SPECTER (like almost every embedding model) only "reads" about **512 tokens** (≈ 350 to 400 words): anything beyond that it **truncates silently**. If you gave it a 10-page PDF, the vector would represent... the first page. The rest, ignored with no warning. This kind of silent limit is a classic trap of language models.

## The fix: chunk and average

**Concept: chunking + mean pooling**

`embed.embed_texts_mean()` does two steps:

1. **Chunking**: it splits each text into pieces of ∼180 words (comfortably below the limit).
2. **Mean pooling**: it vectorises each piece and computes the **average** of all the vectors → a single vector representing the COMPLETE document.

The geometric intuition: each piece is a point on the map of meaning; the average is the *centre of gravity* of all of them, the mean position of what the document is about.

**Watch out / common mistake**

Averaging also *dilutes*: if the PDF mixes very different topics, the centre of gravity falls "between" them and may not represent any of them well. For task instructions (thematically coherent) it works very well; for miscellany documents you would have to group the pieces by topic first. A known and accepted limitation.

## The real test

Simulated brief: *"technical report comparing collectors for lepidolite flotation and their selectivity against feldspar and quartz gangue"* (repeated until it forced several chunks). Result: a top-5 of pure collector papers, headed by the LCJ collector one (similarity 0.897). The centre of gravity landed exactly where it should.

**In one sentence**

Embedding models truncate at ∼512 tokens silently; task mode dodges this by chunking the text and averaging the vectors (mean pooling = the centre of gravity of the meaning). Tested: a collectors brief returns collector papers.

# Note 27: Recommended granularity: let the data choose (DBCV)

Froth project · learning Machine Learning by building it

## The question that started this

"I want a *set to recommended* button with a REAL statistical reason: not a standard value, but one computed every time." Exactly the right attitude: fixed defaults are opinions; the data changes with every corpus.

## The right metric: DBCV

**Concept: DBCV (Density-Based Cluster Validity)**

To evaluate a *density-based* clustering (HDBSCAN) not just any metric will do: the famous *silhouette*, for instance, assumes round, compact clusters. **DBCV** is the family's native metric: it rewards clusterings whose groups are **dense inside** and **well separated outside**, respecting irregular shapes. In code: `hdbscan.HDBSCAN(gen_min_span_tree=True).relative_validity_`.

**Concept: the sweep**

`cluster.recommend_min_cluster_size()` tries EVERY granularity from 3 to 20, computes the DBCV validity of each, discards the degenerate ones (a single cluster is not a clustering) and returns the winner plus a diagnostic table. With today's enlarged corpus: recommended = 7 (validity 0.51). If the corpus changes tomorrow, it recomputes by itself (cached by data version).

**Watch out / common mistake**

The recommendation is a *statistical optimum*, not a final truth: it optimises density and separation, not "which story serves the expert". That is why the slider is still there, wandering is allowed and the button always brings you back. Machine proposes; human disposes (the pattern of all of Froth).

**In one sentence**

The "Use recommended" button does not store a magic number: it sweeps granularities and picks the one maximising DBCV, the native quality metric of density-based clustering, recomputed for each corpus. Real statistics, with the slider free for exploring.

# Note 28: Multi-source connectors: your institutional account joins the game

Froth project · learning Machine Learning by building it

## Why OpenAlex alone is not enough

OpenAlex is huge (∼250M works) but it does not have everything, and sometimes it is missing abstracts. The idea: let the user **connect their own accounts** ("Connect with Google" style) and see papers the free database does not bring.

## The connectors (froth/sources.py)

| Source | Requires | What it adds |
|---|---|---|
| OpenAlex | nothing (key optional) | the BASE: abstracts + references |
| Crossref | just an email | massive publisher metadata |
| Semantic Scholar | FREE key (optional) | abstracts + open access PDFs |
| Scopus (Elsevier) | institutional key | Elsevier's curated catalogue |
| ResearchGate | **NO**: no public API; its ToS forbids scraping | |

Design principles:

- **OpenAlex rules**: it is the preferred record (the richest); the extras only ADD papers we did not have.
- **Dedup by DOI and then by normalised title**: the same paper arrives through several sources and must count once.
- **Soft failure**: if a source is down or rate-limits you, it says so and the pull carries on with the others. A connector never takes the harvest down.
- **The keys are yours and local**: they live in `.froth_keys` (gitignored). On the public website they will live only in your browser session. Froth never holds other people's keys on a server.

## The first multi-source harvest (real numbers)

- Crossref: 227 raw results → the dedup recognised **58 duplicates** of OpenAlex and contributed **169 new ones** (after the abstract filter: corpus 126 → **157**).
- Semantic Scholar without a key: immediate rate limit → it **failed softly** exactly as designed (with the free key it is stable).
- The enlarged corpus revealed an **emerging subtopic**: bubble and fine-particle physics (+ bastnaesite), buried before, now a cluster of its own. More data → more resolution on the map.
- The relevance filter discarded only 6: the extra harvest came in clean. The self-washing of Note 17 scaled without being touched.

In one sentence

sources.py adds Crossref/S2/Scopus to OpenAlex with DOI-then-title dedup and soft failure; the keys are local and the user's (the "Connect institutional account" panel). First harvest: +31 net papers and a new subtopic emerged. ResearchGate is out by design (no legal API).

# Note 29: Phase 6: generalising is not training (and the multi-topic engine)

Froth project · learning Machine Learning by building it

## The clarification that opens the phase

The plan: "iterate over each of the cohort's 22 titles and keep training". Right in spirit, but those are **two different acts**:

**Concept: generalising (6.1 to 6.3) vs. training (6.5)**

- **Iterating the 22 titles trains NOTHING**: the model (SPECTER) does not change a single weight. What gets hardened is the **code**: every odd title that breaks something forces a fix. It is the 1→2→22 ladder from the original plan.
- **Really training** (6.5) does change the model's weights: fine-tuning SPECTER with the thousands of abstracts accumulated from the 22 corpora. PyTorch, loss, epochs, on the RTX 5060 GPU (confirmed capable).

The two acts feed each other: generalising produces the training dataset.

## The multi-topic engine (6.1)

Until yesterday the topic lived *nailed* into `config.py`. Now:

**Concept: config.set_topic(title)**

One call re-points the WHOLE engine: it derives new search terms and moves the data paths into a folder of their own (`2_Datos/topics/<slug>/`). It works because each module reads `config.*` *at execution time*, not at import time, a design detail that paid dividends. (A classic trap caught: `def f(x=config.TOPIC)` freezes the value at import; it was corrected to `x=None` plus resolving inside.)

**Concept: derive_search_terms(): from title to searches**

Topic 1's terms were written by hand. For ANY title, the heuristic in `froth/terms.py`: it strips filler words ("innovative approaches for the..."), keeps **real phrases from the title** (consecutive bigrams and trigrams: *froth flotation*, *scheelite ores*, *Quantitative XRD*), respects **acronyms** (XRD, LIBS, NiMH) and extracts the **content of brackets** (that is where the elements and deposits live). Deterministic, no dependencies. The local LLM layer (Ollama) will come on top as an optional refiner, a decision already taken: local, private, free.

## The 1→2 milestone (tested)

The full pipeline on *"scheelite ores by froth flotation"* **without touching code**: 47 papers, its own folder, and clusters that make metallurgical sense (Falcon concentrator, SNF reagents, frothers). And the test paid off immediately:

Watch out / common mistake

**What the 2nd topic taught** (this is why the ladder exists):

1. The keyword "nbsp" gave away that Crossref abstracts carry **HTML entities** ( ). The cleaner now does `html.unescape`.
2. The `.gitignore` did not cover the new per-topic folders. The data nearly ended up on GitHub.
3. On small corpora (47 papers) a fixed granularity of 8 leaves too much noise → the pipeline now accepts `min_cluster_size="auto"` (chosen by DBCV, Note 27, per corpus).

Three real fixes from ONE new title. The 22 in the batch will find more, and that is exactly their job.

## The parade (6.2)

`froth/batch.py` reads the 22 cohort titles from the CSV and runs the pipeline for each one, with a robustness report per topic (papers, relevant, clusters, noise, granularity chosen, seconds, status). Defensive design: the report is saved after **every** topic and later runs **resume** where they left off (a rate limit loses nothing). Failures do not stop the parade: they stay as FAILED in the table, and that table is the fix-it agenda for 6.3.

In one sentence

Iterating titles hardens the code (generalising); changing weights is training (6.5). The engine is already multi-topic (set_topic + derived terms + per-topic folders), tested with scheelite ores, which threw in 3 fixes along the way. The batch of 22 runs with resume and a per-topic report: its failure table is the work plan for 6.3.

# Note 30: The birth of specter-mineral: your first real training run

Froth project · learning Machine Learning by building it

## What happened (in 83 seconds)

Your RTX 5060 took generic SPECTER and **fine-tuned** it with the ∼1,250 title↔abstract pairs of the master corpus: 2 epochs, 160 steps, batches of 16. The result lives in `models/specter-mineral`, an embedding model that speaks the dialect of mineral processing. On CPU it would have been hours; on your GPU, 83 seconds.

## The concepts you have just used

**Concept: fine-tuning ≠ training from scratch**

We did not build a new model: we took SPECTER's weights (which already know how to "read papers") and **nudged them gently** towards your domain. Like a generalist geologist taking a specialisation, they do not go back to primary school.

**Concept: the elegant move: self-supervision with MNRL**

Training usually demands human labels ("these two texts are alike"). We labelled NOTHING. The trick: **MultipleNegativesRankingLoss**. In each batch of 16 papers, a paper's title must end up closer to *its own* abstract than to the other 15 in the batch. Free positives (a paper's title and abstract talk about the same thing) and free negatives (the rest of the batch). A flat corpus becomes a training dataset without lifting a finger.

**Concept: the vocabulary of training**

- **loss**: the number measuring "how wrong the model is"; training = pushing the weights to bring it down.
- **step**: processing one batch and adjusting the weights once. You saw "160 steps".
- **epoch**: one complete pass over the whole dataset. We did 2.
- **warmup**: the first steps with very gentle adjustments, so as not to wreck what the model already knew.
- **mixed precision (amp)**: arithmetic in lower-precision numbers where it does not matter, part of the reason for the speed on GPU.

**Watch out / common mistake**

**The stumbles teach too**: the training failed TWICE before it started. First `datasets` was missing, then `accelerate` (the Hugging Face Trainer's kit). Ecosystem lesson: in modern ML, "installing the library" is rarely enough; the training stack is its own beast. Both went into requirements.txt for good. And before that: pip refused to swap torch CPU→CUDA because "the version was already satisfied" (same number, different variant), solved with `--force-reinstall --no-deps`.

## How we judge whether it was worth it (6.6)

A new model is not better just for existing. `froth/compare.py` puts BOTH models through the same exam over the 22 topics: same corpora, same UMAP (fixed seed), granularity chosen by DBCV. Who produces more valid clusters? It also exports a **blind test**: two A/B cluster listings with the key hidden, so the expert (you) can judge without knowing which is which. If specter-mineral does not win, that is ALSO an honest and publishable result.

In one sentence

Fine-tuning = pushing existing weights towards your domain; MNRL turns the corpus into a dataset with no labelling (your title must love your abstract more than other people's); 83s on your GPU after two dependency stumbles. The verdict comes from DBCV over 22 topics and your blind test, not from enthusiasm.

# Note 31: The fine-tuning verdict: specialisation, not magic

Froth project · learning Machine Learning by building it

## The experiment (6.6)

Two models competed over the cohort's 22 corpora with the SAME exam (same data, same UMAP seed, granularity chosen by DBCV): generic SPECTER against **specter-mineral**, your model fine-tuned in 83 seconds of GPU with ∼1250 title↔abstract pairs.

## The numbers (the story is in the variance)

- **specter-mineral won 14 of 22 topics** on DBCV validity.
- But the *mean* delta was +0.007 with $\sigma = 0.170$, a flat average hiding a strong pattern:
- **It won big in the core of the domain**: hydrometallurgical recycling (+0.30), XCT slags (+0.24), Beauvoir core imaging (+0.20), your thesis (+0.05).
- **It lost at the periphery**: geopolymer mortars (−0.43), refinery simulation (−0.27), AI on conveyor belts (−0.21).

**Concept: the lesson: a small fine-tune specialises towards the centre of mass**

With little data, fine-tuning does not "improve the model in general": it **drags it towards where the mass of the training corpus is** (flotation/geology). Topics near that mass gain resolution; distant ones lose it. Open hypothesis (your natural next experiment): does the 2× corpus with Scopus reduce the peripheral penalty?

## The double blind (your moment)

Two cluster listings from the Beauvoir corpus, labelled A/B at random, key hidden. You chose **B** "because it separated fines flotation from lepidolite collector chemistry", and B was **your model**. Metric and expert agreed on the domain you master: the strongest validation available.

**Watch out / common mistake**

A "no improvement on average" with clear structure (wins here, loses there, and we know why) is a BETTER scientific result than an unexplained "+2% everywhere". Do not inflate: qualify.

**In one sentence**

14 wins out of 22, mean delta ≈ 0, large $\sigma$: a small fine-tune specialises towards the core of the corpus at the cost of the periphery, confirmed by the metric (DBCV) and by a double-blind test where you picked your own model without knowing it.

# Note 32: h-index per cluster: how many papers are "must-reads"

Froth project · learning Machine Learning by building it

## The problem (your own no-magic-numbers rule)

In the reading guide we wanted to mark the must-reads of each subtopic. Top 3? No: "top 3" is a magic number, just as arbitrary for a cluster of 110 geology papers as for one of 9 on tomography. You asked to *research how to research* it: a threshold computed **per corpus and per cluster**.

## The idea stolen from bibliometrics: the h-index

The h-index was invented to measure researchers, but the definition works on ANY list of citation counts:

**Concept: h-index of a cluster**

The largest $h$ such that the cluster has $h$ papers with at least $h$ citations each. You sort by descending citations and look for the point where the rank overtakes the citations: right there the evidence stops holding itself up.

Why it is elegant: **the cluster sets its own yardstick**. A heavily cited cluster demands hundreds of citations to enter the must-read list; a young one settles for dozens. Zero parameters to tune.

## The real Beauvoir data

- Geology/granites (110 papers) $\rightarrow h = 35$: mature literature, the bar set itself very high.
- XCT tomography (9 papers) $\rightarrow h = 3$: a young niche, a proportional bar.
- USGS reports (mostly with 0 to 2 citations) $\rightarrow$ the h-index degenerates ($h > n/2$, it would mark "almost everything is a must-read"): that is where the **head/tail breaks fallback** comes in. It cuts at the cluster's mean citation count, which in long-tailed distributions separates the heavy head from the rest ($\rightarrow 7$).

**Watch out / common mistake**

Citations $\neq$ quality: they measure accumulated attention and they punish the recent (the 59 papers from 2025 to 2027 with 0 citations are fresh research, not bad research). The must-read says "this is the canon the field expects you to have read", not "this is the only thing worth anything".

## Where this rule lives (and a lesson from 2026-08-03)

Until today this note described a fallback that had **never been implemented**: the h-index was in the code, the mean cut was not. It was implemented while building the Obsidian export, in `froth/export_obsidian.py` (`_must_reads` and `_mean_cut`).

Watch out / common mistake

**A small deviation broke the whole rule.** While programming it, Claude repeated the mean cut *up to six times* instead of doing it ONCE, thinking that would be more selective. Measured effect on your thesis corpus: the 340-paper subtopic holds a classic with 4,250 citations that drags the mean, and repeating the cut strangled the list down to **1 paper**. With a single cut it gives 36. In total, the repeated version left 2 papers or fewer in **20 of the 29 subtopics with 30+ papers**; this note's rule does that in none of them.
The lesson: when a note fixes a method, the code has to follow it *literally*. "I made it a bit stricter" is not an improvement, it is a different method, and it has to be measured before being called better.

Current figures with the correct rule: thesis corpus, 3,867 papers in 104 subtopics → 893 must-reads (median 6 per subtopic, none left short).

In one sentence

Must-reads with no magic numbers: an h-index per cluster (self-calibrated to each subtopic's maturity) with a head/tail breaks fallback for when the citations are too flat for the h-index to discriminate.

# Note 33: Calibrated springs and frontier papers: what position means in the network

Froth project · learning Machine Learning by building it

## Your question: "do the springs mean anything?"

In the Network view the nodes arrange themselves with a **force layout** (forceAtlas2): each edge is a spring pulling things together, and all the nodes repel each other like charges. The equilibrium of that tug of war decides the position. By default vis.js uses the SAME spring length for every edge, so visual distance was pure aesthetics.

## The calibration: length = dissimilarity

Our edges already carry a weight $w$ = cosine similarity (with a 0.75 threshold). Now each spring is made to measure:

$$\text{length} = 60 + 420\,(1 - w)$$

Nearly identical neighbours ($w \to 1$) end up ~60 px apart; the ones that barely passed the threshold, ~165 px. Since that change, **close on screen ≈ close in meaning**. The network answers the same question as the UMAP map, but with the connections made explicit.

## The mystery of the "yellow node stuck to the blue hub"

You noticed nodes of one colour sitting inside another's territory. It was not a bug: they were **frontier papers**. The chain of events:

1. HDBSCAN cuts the clusters on the **2D** UMAP map.
2. The network connects neighbours in the original **768D**.
3. A paper can fall on the yellow side of the 2D cut while having almost all its semantic neighbours in the blue cluster → the springs drag it towards the blue.

**Concept: frontier paper detection**

For each node we count its neighbours' clusters. If ≥ 60% belong to ANOTHER cluster, it is a frontier paper: it is drawn with a **dashed ring in the neighbouring cluster's colour** and the tooltip declares it ("bridges into the neighboring subtopic"). In Beauvoir: 3 out of 237 papers.

**Watch out / common mistake**

Frontier papers are not clustering errors: they tend to be the interdisciplinary ones, exactly those connecting subtopics. For a reader hunting research gaps, they are among the most interesting on the map.

In one sentence

Springs with length $= 60 + 420(1 - w)$: visual proximity now IS semantic distance. The "misplaced" nodes are frontier papers (most of their neighbours in another cluster), now marked with a dashed ring in the neighbour's colour.

# Note 34: Extractive vs abstractive summarising: why ours cannot hallucinate

Froth project · learning Machine Learning by building it

## The Phase 7 problem

Review v1 is an honest but dry scaffolding: keywords + lists of papers. For it to actually *say* something there are two routes, and the difference between them is THE most important design decision of this phase.

**Concept: extractive vs abstractive**

**Abstractive**: a model GENERATES new text that "sounds like" a summary (ChatGPT, Elicit). Fluent, but the text exists in no source: it can invent statements and, worse, references.
**Extractive**: the system SELECTS real sentences that already exist in the abstracts. Less fluent, but every sentence has a source paper: the citation is traceable *by construction*. Hallucinating is impossible: not a single word is generated.

## Why we chose extractive first

- **Trust**: an academic review with ONE invented reference is dead. Plain ChatGPT is famous for citing papers that do not exist.
- **Traceability for free**: since the sentence comes from an abstract in the corpus, the "know more" button and the [n] citation already work with no extra verification.
- **It is real ML**: ranking sentences uses embeddings, cosine, graphs and PageRank, everything you already built, at a new scale (sentences, not papers).
- Polishing with an LLM (paraphrasing sentences ALREADY chosen, verifying citations) stays as an OPTIONAL layer on top, never as the base (7.6).

## The kitchen: from abstract to usable sentences

In `froth/review.py`, `split_sentences()` cuts each abstract where there is final punctuation followed by a capital letter, and `_self_contained()` discards whatever does not survive on its own: fragments (<8 words), marathons (>60) and sentences starting with **anaphora**, "This method...", "It also...", because they point at a text the review's reader will never see.

**Watch out / common mistake**

The extractive approach inherits the limitations of its raw material: we work with ABSTRACTS (not full text), and a badly written abstract yields mediocre sentences. Review v2 is still an improved scaffolding, not final prose. Honesty above all.

In one sentence

Extractive = selecting real sentences (traceable, no hallucination); abstractive = generating text (fluent, risky). Froth builds on extractive and leaves the local LLM as an optional polisher of already-chosen sentences.

# Note 35: The centroid sentence: the relevance filter in reverse

Froth project · learning Machine Learning by building it

## The idea (you already knew it without knowing)

In Phase 3, the relevance filter measured each paper's cosine against the TOPIC vector and *discarded* whatever was far. The centroid sentence uses the SAME mathematics with the opposite intention: measure each sentence's cosine against the cluster's centre and *crown* whatever is closest.

**Concept: centroid and representative sentence**

The **centroid** of a cluster is the average of its papers' vectors (768D): its "semantic centre of mass". The abstracts' sentences are vectorised with the SAME model (SPECTER), so they live in the same space and are comparable. The sentence with the highest cosine to the centroid is the one that best sums up what the subtopic is about, chosen by geometry, not by opinion.

## The real results (Beauvoir corpus, 237 papers)

`python -m froth.review` on your data:

- Geology cluster ("ma, melt, compositions"): score **0.947** for *"LCT-type granitic pegmatites represent a major global source of lithium..."*, a textbook definition of the subtopic.
- USGS cluster ("usgs, old scrap, year"): *"Minerals Yearbook, These annual publications review the mineral industries of the United States..."* The keywords never told you that cluster is ANNUAL REPORTS; the sentence does.
- Lepidolite flotation cluster: *"Efficient flotation of lepidolite is crucial for the extraction and recycling of lithium resources..."*

This settles your request for meaningful titles (7.1b): keywords to classify, **centroid sentence as a subtitle** to understand.

## Two guards against traps

- **Per-paper cap** (`per_paper_cap=2`): without it, a very "central" abstract could monopolise a cluster's entire summary.
- The centroid is computed over the PAPER vectors (the ones on the map), not over the sentences: that way the summary is faithful to the same space the user is looking at.

**Watch out / common mistake**

The top sentences resemble the centroid... and therefore they resemble EACH OTHER: centroid ranking tends towards redundancy (three sentences saying the same thing). That is exactly the problem MMR attacks in sub-phase 7.3.

In one sentence

Centroid sentence = cosine(sentence, cluster centre), the same mathematics as the relevance filter but to crown instead of discard. On Beauvoir it produces textbook definitions with scores of 0.92 to 0.95. Its weakness (redundancy) is corrected by MMR (7.3).

# Note 36: TextRank: PageRank over sentences (the hub among sentences)

Froth project · learning Machine Learning by building it

## The idea: recycling Google's algorithm

PageRank ordered the web with a circular but solvable idea: *a page matters if important pages link to it.* TextRank (Mihalcea & Tarau, 2004) swaps pages for sentences and links for similarity: *a sentence matters if important sentences resemble it.* It is EXACTLY your hub concept from Phase 4, applied inside each cluster.

**Concept: the sentence graph and the random walk**

Nodes = the cluster's sentences; edge weight = cosine between their SPECTER vectors. PageRank is computed by **power iteration**: imagine a reader hopping from sentence to sentence, preferring hops towards similar sentences; with probability $1 - d$ ($d = 0.85$) they teleport to any one at random. After iterating

$$r \leftarrow \frac{1-d}{N} + d\,W r$$

some 50 times, $r$ converges: the time the reader spends on each sentence IS its importance. In `froth/review.py` it is 4 lines of numpy, with no new libraries.

## Centroid vs TextRank: they do not measure the same thing

- **Centroid** rewards the *typical*: the sentence most similar to the average.
- **TextRank** rewards the *endorsed*: the sentence with the most consensus "votes", even if the consensus comes from a dense subgroup.

On your corpus (run `python -m froth.review` and judge for yourself):

- USGS cluster: BOTH pick "Minerals Yearbook, These annual publications review..." When two independent metrics agree, that sentence IS the topic.
- Lepidolite cluster: TextRank preferred *"...the lepidolite flotation process and its challenges are summarized, including poor selectivity..."* REVIEW sentences (they summarise the field's challenges) get many votes because they touch everything. For a literature review, that is a virtue.
- Geology cluster: TextRank went off to exogenous deposits. A dense subgroup of mineralogical sentences votes for itself and captures the ranking. The centroid, anchored to the PAPERS' vectors, resists that trap better.

**Watch out / common mistake**

The scores are not comparable between methods: cosine lives in 0 to 1; PageRank splits a total probability of 1 among N sentences (which is why you see 0.008). Compare RANKINGS, not numbers. And neither of them avoids redundancy, that is MMR (7.3).

In one sentence

TextRank = PageRank over the sentence similarity graph, solved by power iteration. It rewards consensus (good for summary sentences), but a dense subgroup can capture it; the centroid rewards the typical. The expert decides which reads better, or they get combined.

# Note 37: MMR: relevant but different (and an objective that came out degenerate)

Froth project · learning Machine Learning by building it

## The problem that remained

The centroid (note 35) has a structural flaw: sentences resembling the centre resemble EACH OTHER. In your geology cluster, the 3 best sentences had a mutual similarity of **0.945**, a summary saying the same thing three times.

**Concept: Maximal Marginal Relevance (Carbonell & Goldstein, 1998)**

Greedy selection, one sentence at a time. Each candidate is scored as

$$\lambda \cdot \text{relevance} \; - \; (1 - \lambda) \cdot \text{redundancy}$$

where redundancy = its maximum similarity to what is ALREADY chosen. With $\lambda = 1$ you are back to the pure centroid; lowering $\lambda$ buys diversity by paying in relevance. The first one chosen is always the most relevant (nothing has been chosen yet to be redundant with).

## The stumble (which teaches more than the success)

First attempt at choosing $\lambda$ from the data: "maximise relevance − redundancy". Result: it recommended $\lambda = 0.5$... the EDGE of the grid. That objective is **degenerate**: lowering $\lambda$ ALWAYS reduces redundancy faster than it loses relevance, so the "optimum" runs off downwards indefinitely. A recommender that always picks the extreme is measuring nothing.

## The rule that does work

Looking at your real sweep (Beauvoir corpus, 12 clusters):

| $\lambda$ | relevance | redundancy |
|---|---|---|
| 0.5 | 0.868 | 0.758 |
| 0.6 | 0.906 | 0.829 |
| **0.7** | **0.918** | **0.860** |
| 0.8 | 0.921 | 0.873 |
| 1.0 | 0.923 | 0.884 |

From 1.0 down to 0.7 relevance barely falls (−0.5%) and the diversity comes for free; from 0.7 downwards you start really paying. Rule adopted (interpretable): **the most diverse $\lambda$ that keeps $\geq$ 99% of the achievable relevance**. Your corpus voted $\lambda = 0.7$ on its own, and it coincides with the literature's default, but now we KNOW why it holds for this data (project rule 9).

**Try it yourself (hands on the keyboard)**

`.venv\Scripts\python.exe -m froth.review`: at the end it prints the sweep and the before/after of the most redundant cluster. The centroid gives 3 cloned pegmatite sentences; MMR keeps the definition and adds Nb-Ta granites and the Erzgebirge. Three angles, not an echo.

**Watch out / common mistake**

MMR is greedy: it picks the best NOW without looking ahead, because the exact optimum is combinatorial (trying every combination of sentences). For 3 to 5 sentence summaries, the greedy approximation is the standard and it is more than enough.

**In one sentence**

MMR $= \lambda \cdot$relevance $- (1-\lambda) \cdot$redundancy, greedy selection. The first objective for choosing $\lambda$ came out degenerate (it stuck to the edge); the final rule, maximum diversity keeping 99% of the relevance, chose 0.7 with your data. That completes the 3 ranking pieces: typical (centroid), endorsed (TextRank) and diverse (MMR). Next comes assembling draft v2 (7.4).

# Note 38: Draft v2: blending judges with Borda and citing by construction

Froth project · learning Machine Learning by building it

## Assembling without inventing

`draft_review_v2()` puts the three pieces (notes 35 to 37) together into a per-cluster draft: one sentence OPENS (the panorama), 2 sentences BACK IT UP (typical but diverse), and each carries its [$n$] citation pointing at the paper it came from verbatim. The numbering is global and in order of appearance; the same engine emits the `.bib` for LaTeX/Zotero. Nothing is generated: nothing can be hallucinated.

## The opener problem and the Borda solution

The plan was "TextRank opens" (its panorama sentences are ideal). But in geology it opened with the exogenous deposits sentence, the dense-subgroup capture from note 36. The centroid scores (cosine 0 to 1) and TextRank's (a distributed probability) are NOT comparable, so they cannot be averaged... but the *rankings* can.

**Concept: Borda count (1770, elections; today, rank aggregation)**

Each method orders the sentences and the position is the "vote": 1st = 0 points, 2nd = 1, and so on. The sentence with the LOWEST sum of positions wins. To hijack the opener you would have to fool BOTH judges at once: the dense subgroup loses because the centroid ranks its sentences low. With your data: geology went from exogenous deposits to the chemical composition of pegmatites; lepidolite kept its panoramic opening.

## Fidelity: reject, do not trim

The first draft opened lepidolite with *"In addition, ..."*, a connector pointing at text that does not exist. Two possible fixes: TRIM the connector (which modifies the verbatim quote: forbidden by the project's fidelity rule) or REJECT the sentence and let Borda promote the next one. We reject: the `_self_contained()` filter now also vetoes two-word discourse connectors ("In addition", "On the other hand", "In contrast"...). The next one chosen was *"Efficient flotation of lepidolite is crucial..."*, a better opening with zero manipulation of the original.

**Watch out / common mistake**

The † in the draft marks sentences where the centroid and TextRank agree in the top 3 (consensus = a seal of confidence). If a section has no †, nothing is wrong, it only means the two judges saw different virtues.

**Try it yourself (hands on the keyboard)**

`.venv\Scripts\python.exe -m froth.review` generates `3_Resultados/review_draft_v2.md` (42 traceable refs on Beauvoir) and its `.bib`. Open the .md and verify a citation: find the [$n$], go to References, and check that the sentence exists in that paper's abstract. Auditable traceability.

In one sentence

Draft v2 = Borda picks the opener (fooling two judges is harder than one), MMR supplies the diverse backup, every sentence goes in verbatim with an $[n]$ citation + BibTeX. Sentences with dangling connectors are REJECTED (never trimmed): fidelity rules.

# Note 39: The LLM as a verified polisher (never as an author)

Froth project · learning Machine Learning by building it

## The hierarchy of trust

Draft v2 is extractive: guaranteed truth, so-so prose. An LLM writes excellent prose... with no guarantee of truth. The architecture of phase 7.6 takes the best of both WITHOUT inheriting the risk:

**Concept: polishing ≠ generating**

The local LLM (Ollama) receives ONE narrow instruction: rewrite a section's paragraph so it flows, without adding or removing statements, keeping every $[n]$ citation exactly once. It does not research, it does not opine, it does not cite: it POLISHES sentences that already exist. The knowledge comes from the corpus; the LLM only contributes grammar.

## The verification gate (the heart of the design)

Trusting is not enough: every rewritten section goes through `_verified()` in `froth/polish.py`:

- **Citation census**: the $[n]$ citations in the polished text must be EXACTLY those of the original (one fewer = it deleted evidence; one more = it invented).
- **Length band** (0.5 to 1.7×): outside that, it either over-summarised (losing statements) or rambled (probably adding).
- If ANY check fails → the section keeps its extractive original, silently. The draft can only improve, never lie.

Tested without a server: the gate rejected a deleted citation, an invented citation and shrunk text; and with no Ollama running the draft stays intact (soft failure, like the source connectors).

## Why LOCAL (Ollama) and not an API

A decision of yours from Phase 6 that pays off here: free (no quotas), private (your review never leaves your PC), offline, and a differentiator, since Elicit/SciSpace/Consensus depend on OpenAI. Your RTX 5060 runs an 8B model comfortably. Setup in `docs/OLLAMA_SETUP.md` (one installer plus `ollama pull llama3.1:8b`, ∼5 GB, once only).

**Watch out / common mistake**

Temperature 0.2 (almost deterministic): for polishing you do not want creativity, you want obedience. And the polished version is a SEPARATE DOWNLOAD, the extractive one stays the visible default. The chain of trust: measured map → real sentences → verified polished prose. Every link auditable.

In one sentence

The local LLM rewrites paragraphs that are already built and an automatic gate (citation census + length band) discards any suspicious rewrite, section by section. LLM prose with the traceability of the extractive one, or nothing.

# Note 40: Cluster titles that mean something: KeyBERT (not trimmed sentences)

Froth project · learning Machine Learning by building it

## The problem (you spotted it)

Version 1 of the subtitles took the cluster's most central *sentence* and cut it at 12 words. Result: headings like "Considering both…", "In the present st…", fragments that hint at the topic but feel chopped off, like a half-read page. And the old keywords (c-TF-IDF: "dpa, dda, lepidolite") were no better: they classify well but they *do not communicate*.

## How the field solves it: KeyBERT

The standard (KeyBERT, and BERTopic's representation layer) does not trim sentences: it extracts short **keyphrases** and ranks them by MEANING.

**Concept: KeyBERTInspired in 3 steps**

**1. Candidates**: 2 to 3 word n-grams from the cluster's text (with the same hardened vectoriser from `cluster.py`: no digits, chemical elements spared).
**2. Rank**: embed each candidate with SPECTER and measure its cosine to the cluster's *centroid*.
**3. Diversify**: MMR so the chosen phrases are not nearly identical.

The elegant part: these are the SAME pieces Froth already had (SPECTER + cosine + centroid + MMR from Phase 7). That is why it was built by hand, with no new dependency, the project's pattern: build each piece in order to understand it.

## Two stumbles that teach (your real data)

The first attempt produced titles with OCR noise ("limu", "nmic", "cgfs") and generic ones ("ore flotation" where it should have said "lepidolite flotation"). Two fixes:

- **`min_df` $\geq$ 2**: require the phrase to appear in SEVERAL papers of the cluster. OCR rubbish lives in a single abstract $\rightarrow$ out.
- **Contrastive score** (the c-TF-IDF idea, but in embedding space):

$$\text{score} = \cos(\text{phrase}, \text{centroid}_c) - 0.4\ \cos(\text{phrase}, \text{centroid}_{\text{global}})$$

  It rewards what is close to THIS cluster and far from the whole corpus. That is how "lepidolite flotation" (specific) beats "ore flotation" (generic, close to the centre of everything).

**Watch out / common mistake**

The keyphrase closest to the centroid is NOT the best title: it tends to be the most generic. Specificity (far from the global average) is what makes a title *say* something. Same lesson as the h-index and the relevance filter: the signal is in the contrast, not in the absolute value.

**Try it yourself (hands on the keyboard)**

The short title goes in the heading; the full centroid sentence is stored in the `summary` column and shows up on **hover**: short on top, detail on mouseover, exactly as you asked.

**In one sentence**

Cluster titles = 2 to 3 word keyphrases extracted KeyBERT-style (candidates + SPECTER + cosine to the centroid + MMR), with `min_df` against the noise and a contrastive score (cluster − global) to prefer the specific. Short, meaningful, with no trimmed sentences; the full sentence lives in the hover.

# Note 41: The re-fine-tune verdict: more data did NOT help

Froth project · learning Machine Learning by building it

## The hypothesis (your doubt from the learning log)

Fine-tune v1 (specter-mineral, 1,290 papers) won 14 of 22 on DBCV but punished the periphery (down to −0.43). Natural hypothesis: with **3× more data** (a re-harvest with keys → 4,045 papers) the v2 model should generalise better and stop penalising the distant topics. We put it to the test, measured.

## The clean experiment

We trained v2 (4,045 title↔abstract pairs, MNRL, 249s on your RTX 5060) and compared **the three models over the SAME 22 new corpora** by DBCV: base (generic SPECTER), v1 (the old fine-tune) and v2 (the big fine-tune).

| | beats base | mean delta |
|---|---|---|
| v1 (1,290 papers) | **16/22** | **+0.093** |
| v2 (4,045 papers) | 14/22 | +0.069 |
| v2 vs v1 head to head | 10/22 | **−0.024** |

**Concept: the result: scaling the data did NOT help (and slightly hurt)**

v2 beats base FEWER times than v1 (14 vs 16) and by a smaller margin. Head to head, v2 beats v1 in only 10 of 22 with a negative mean delta. **The small, curated fine-tune was as good as or better than the big one.**

## Why (the real lesson)

The 3× re-harvest brought more papers, but also more DIVERSE and NOISIER ones (multi-domain term collisions like "attrition"/"autogenous", more subdomains). Self-supervised fine-tuning drags the model towards the *centre of mass* of the training corpus (note 31); enlarging the corpus with noise moves that centre towards the generic and the specialisation gets DILUTED.

**Watch out / common mistake**

In self-supervised fine-tuning, the QUALITY and coherence of the corpus matters more than the quantity. "More data" is a reflex, not a law. The variance is still enormous: v2 ranges from +0.47 (XCT slags) to −0.51 (iron silicate fines). The fine-tune is a per-topic bet, not a uniform improvement.

## What stays in production

Nothing changes: production continues with **base** SPECTER (the most stable across topics). The fine-tunes are the RESEARCH RESULT. v1 is kept (`models/specter-mineral-v1`); v2 remains as a reference. Full evidence in `2_Datos/model_comparison_3way.csv`.

In one sentence

Re-fine-tune with 3× the data: v2 beats base 14 of 22 (v1 was winning 16 of 22), and loses to v1 head to head (mean delta −0.024). More data did NOT help, the big noisy corpus diluted the specialisation. An honest thesis lesson: in self-supervised fine-tuning, corpus curation > size. “No improvement” is also a result.

# Note 42: When the program does not crash but HANGS: watchdogs and subprocesses

Froth project · learning Machine Learning by building it

## The real accident (the early hours of 14 July)

We launched the uncapped re-harvest of the 22 topics. The batch ran perfectly: 18 giant corpora (4,000 to 16,681 papers each) in about 3 hours… and then, silence. The next morning: **9 hours frozen** on topic #20, no error, no message, no progress. The process was still "alive" (77 MB of RAM, zero CPU), waiting for a network response that never came.

## The central lesson: a hang is NOT a crash

Our `run_topic` was already "failure proof": it wrapped the whole pipeline in `try/except`, and every failure became a `FAILED` row in the report so the next topic could carry on. It sounded armoured. It was not.

Concept: crash vs. hang

A **crash** raises an exception: `try/except` catches it and life goes on. A **hang** raises NOTHING: the program simply waits forever (a dead socket, a runaway computation, a blocked driver). No `try/except` in the world interrupts an infinite wait. For that you need someone WATCHING FROM OUTSIDE with a clock: a **watchdog**.

And since the topics ran in series, one single hang froze the entire batch. The probability was not negligible: 22 topics × 4 APIs × thousands of requests.

## The fix: child process + wall clock

Each topic now runs in its own **subprocess** (`python -m froth.batch --one "<title>"`) watched by the parent with `subprocess.run(..., timeout=TOPIC_TIMEOUT_S)` (2,400 s, generous: the slowest legitimate topic took 38 min). If the child goes past the clock, Windows KILLS it, the parent writes `FAILED: timed out` and moves on to the next. The hang went from "a silent 9-hour catastrophe" to "an ugly row in the CSV".

Along the way we also bounded the two computations that really could run away: the DBCV sweep for automatic granularity is $\mathcal{O}(n^2)$ and repeats 18 times, which over 16,000 points is hours. Now, past a cap (`DBCV_SUBSAMPLE_CAP` = 5,000) the sweep runs on a random subsample (the CHOICE of granularity is stable; the cost is not), and the final clustering does use every point. And the abstract backfill, the only network loop with no budget, got its time limit.

## Bonus 1: the Beauvoir plan B

The topic "Recovery of by-products of lithium from the a rare-metal granite" harvested **0 papers**: its derived terms revolved around "a rare-metal granite", too niche for the APIs. Fix: if the harvest comes back EMPTY, it retries ONCE with a broad high-recall query (the first content words of the title: `recovery by-products lithium`). Real result: from 178 old papers to **4,937**.

## Bonus 2: the external enemy (WDAC in waves)

The corporate application control policy (WDAC) intermittently blocks the native DLLs of scipy/hdbscan/torch ("An Application Control policy has blocked this file"). Measured live: it let one child work for 19 minutes and 5 minutes later blocked the other two at import. You do not fight a corporate policy: you WAIT with automated patience. A cheap probe (`python -c "import hdbscan, umap, torch"`) every 5 minutes, and when the wave passes, the heavy work launches. The parent orchestrator, moreover, imports only `config` + `pandas`: with no scientific DLLs, WDAC cannot take it down.

**Watch out / common mistake**

Three different layers of defence, because they are three different failures: `try/except` catches *crashes*, the watchdog with a timeout kills *hangs*, and the window probe waits out *environment blocks*. Confusing them is like believing a life jacket protects you from lightning.

**In one sentence**

The batch froze for 9 hours because a hang raises no exception and nothing was watching. A three-layer fix: one subprocess per topic with a wall-clock timeout (watchdog), bounded computation (DBCV by subsample), and WDAC window probing. Result: 22 of 22 topics with a clear status and ∼169,000 papers harvested; Beauvoir went from 178 to 4,937 with the plan B terms, a synthetic-ore topic from 237 to 9,723. A robust system is not one that never fails: it is one that turns any failure into a row in the report and carries on.

# Note 43: The relevance gate: separating wheat from chaff in 190,000 papers

Froth project · learning Machine Learning by building it

## Glossary in plain words (read this first)

- **Vector**: a paper's numerical "fingerprint", a list of 768 numbers capturing its meaning. Papers about the same thing have similar fingerprints (note 05).
- **Cosine**: the gauge of likeness between two fingerprints. It runs from $-1$ to 1: near 1 = they talk about the same thing; near 0 = nothing in common.
- **Centroid**: the "average" of a group of vectors, its centre of gravity, like the mean grade of a batch of ore.
- **Threshold**: the cut-off value. Above it: the paper enters the map. Below: out.
- **Bimodal**: a distribution with TWO humps. If the good ones make one and the bad ones another, the valley between them is the natural cut.
- **Hub / cluster**: a group of papers very similar to each other, a subtopic.

## The problem YOU found

With the uncapped harvest (190,000 papers), you opened the tailings map and saw *"even human resources stuff, in colour"*. Your hypothesis was exact: no word failed to vectorise, what failed was **the gate** deciding what enters the map.

The old gate: relevance = cos(paper, topic TITLE), with a fixed threshold of 0.55. Measured on tailings: it accepted **93%** of 16,681 papers, including chlamydiosis in koalas (because of biological "iron"), "event management" (because of "management") and "Circular Economy in SMEs" (because of "circular economy"). A short phrase with generic magnets cannot tell facets apart. On top of that, the old ceiling of 25 results per term was an ACCIDENTAL precision filter: removing it (the right decision, coverage is the mission) let in the deep tail of every term.

## Toy mini-example (do the arithmetic with me)

Fingerprints of only 2 numbers (instead of 768). The title points at $T = (1,\ 0)$. Three papers:

| | vector | cos with the title |
|---|---|---|
| A: tailings flotation | $(0.98,\ 0.20)$ | 0.98 |
| B: personnel management | $(0.60,\ 0.80)$ | 0.60 |
| C: koalas | $(0.50,\ 0.87)$ | 0.50 |

With a threshold of 0.55, B enters the map! $(0.60 > 0.55)$. That is your human resources hub.

The **contrastive gate** asks a better question: do you resemble my topic MORE than you resemble the generic?

$$v_2 = \cos(\text{paper},\ \text{core}) - 0.4 \cdot \cos(\text{paper},\ \text{global})$$

where core $\approx A$ (the papers stuck closest to the title) and global = the average of all of them $\approx (0.74,\ 0.67)$ normalised. The arithmetic:

$$v_2(A) = 1.00 - 0.4 \cdot 0.86 = \mathbf{0.66} \qquad v_2(B) = 0.75 - 0.4 \cdot 0.98 = \mathbf{0.36}$$

B ends up far from any reasonable cut. With 768 numbers and 16,681 papers, this same thing produced a **bimodal** distribution with a valley at +0.399, and the threshold is chosen in THAT valley, per corpus (a computed knob, not a fossil constant).

**Concept: the contrastive idea**

It is the same trick that fixed the KeyBERT titles (note 40): reward closeness to the core AND punish closeness to the generic mass. Shared genericness stops counting as merit.

## Stumble 1: the per-paper gate flattened the maps

The first version judged paper by paper… and tailings went from 115 subtopics to **2** (at ANY granularity, the DBCV sweep confirmed it). Why? Keeping only "those similar to the core" leaves a surviving population that is a homogeneous band: a ball with no internal valleys, which HDBSCAN cannot subdivide. We gained precision and destroyed the STRUCTURE, which is half the product.

The fix came out of your own sentence ("an HR hub, *in colour*"): the rubbish arrives in **coherent hubs**, so the gate must judge HUBS: (1) map EVERYTHING, (2) score each cluster by the mean $v_2$ of its members, (3) throw out whole clusters below the valley of those means, (4) loose noise is judged individually. Measured: the particle physics, astronomy, HR and koala hubs died whole; the tailings mortar one did not lose a single paper; 325 domain subtopics intact.

## Stumble 2: your double blind uncovered a bias

You judged 40 borderline cases blind: 62% agreement, but with a direction. You rescued 11 papers the machine was throwing out, almost all about **circular economy**… which is IN your thesis title. Cause: the global centre points partly towards the topic itself (the corpus is full of it), so a paper aligned with the title got a reward from the core AND a punishment from the global: a double penalty for its own identity. Fix: **project out** of the global the direction of the core before punishing. Verified: "Towards Circular Economy Metrics" came back; the koalas and the cheese paper ("Iron Ages"… the Iron Age, not the mineral) stay out.

Closing honesty: re-measured, your agreement against the final gate came out at 60%, practically the same. The fix won the cases you cared about (the circular economy ones), but the $\pm 0.04$ frontier around the valley is ambiguous EVEN for the expert (you let the cheese through because of the "Iron"). We tune no further: that would be overfitting to 40 cases.

**Watch out / common mistake**

The gate's value is at the HUB level (coherent rubbish dies whole, domain structure intact), not in decorating the edge. No one-dimensional frontier is perfect; the right flow is machine proposes → expert validates → the knob is recalibrated where the expert is systematically right (circular economy yes, cheese no).

In one sentence

Old gate: cos with the title + a fixed threshold = 93% accepted, koalas in colour. New gate: a contrastive score (core vs. global WITHOUT the topic's direction) → bimodal → a valley PER corpus → judgement by HUBS to preserve structure. Result: 22 of 22 topics clean with their own threshold, a master set of 77,636 unique relevant papers. Three lessons: (1) contrasting against the generic separates what plain likeness cannot; (2) filtering individuals can destroy the collective structure, judge the group when the signal is a group signal; (3) the human judge finds the biases the metric cannot see, and has their own too.

# Note 44: Citation triplets: training the way SPECTER does, and the inverted verdict

Froth project · learning Machine Learning by building it

## Glossary in plain words

- **Fine-tuning**: taking an already-trained model (SPECTER) and giving it a "specialisation course" with data from YOUR domain.
- **Triplet**: a training example with 3 pieces: the *anchor* (a paper), the *positive* (one that should end up CLOSE) and the *negative* (one that should end up FAR).
- **Hard negative**: a DIFFICULT negative. It resembles the anchor but is not what you are after. It teaches far more than an obvious negative.
- **Epoch**: one complete pass over all the training examples.
- **DBCV**: a quality mark for density-based clustering ($-1$ to 1): it rewards groups that are dense inside and well separated from each other.
- **Double blind**: evaluating without knowing which option is which, so the judgement is not contaminated.

## The question you asked

*"Should we be able to improve the fine-tune, or is SPECTER perfect?"* The short answer: SPECTER is not perfect, but our previous fine-tunes used a weak method. v1 and v2 trained with pairs (title $\leftrightarrow$ abstract): that only teaches "titles go with their abstracts", and it drags everything to the corpus's centre of mass, which is why v2 (3$\times$ the data) did NOT beat v1 (note 41). The original SPECTER is trained differently: with **citation triplets**. If paper A cites B, B should end up close to A; a paper A does not cite should end up far.

## Toy mini-example

Anchor: "Flotation of micas with cationic collectors". Positive: the collector paper THAT IT CITES. Hard negative: a paper cited by the positive but NOT by the anchor (two hops away: a relative of a relative, but not yours). At each step, the training MEASURES the distances and pushes: the positive a little closer, the negative a little further. Repeat 100,000 times and the whole space reorganises itself according to who-cites-whom.

The uncapped harvest made this possible at scale: from 833 triplets (the old corpus) to **150,885** (100,190 with a hard negative), 181$\times$ more signal, because with 190,000 papers many references fall INSIDE the corpus.

## The training run (and its measured knobs)

First attempt: 2 epochs $\times$ 512-token sequences = an estimated 17 hours on your RTX 5060 (3.2 s/step, measured). Your decision: 1 epoch + 256 tokens (abstracts at $\sim$200 tokens fit almost whole) + checkpoints every 1,000 steps (the lesson of note 42: an interruption should cost minutes, not everything). Result: 3.4 steps per second, finished in $\sim$45 minutes. The model: `specter-mineral-v3`.

## The verdict in two rounds

**Round 1, the metric (DBCV, 22 clean corpora, fixed sample of 1,500):**

| | beats base | mean delta |
|---|---|---|
| v1 (pairs, small) | 14/22 | +0.013 |
| v2 (pairs, large) | 9/22 | −0.022 |
| v3 (citation) | 12/22 | **+0.023** |
| v3 vs v2 head to head | 13/22 | +0.046 |

Your question is answered: the right METHOD wins where QUANTITY failed (v2 loses against base; v3 is the only one with a clear mean improvement). And an extra finding: on the old dirty corpora v1 was winning 16 of 22 with +0.093; after cleaning, ALL the advantages shrank. Part of the earlier "improvement" was knowing how to group the rubbish.

**Round 2, the expert (you, double blind, 4 anonymous maps):** you ranked B>D>C>A… which on revealing the key turned out to be **v1** > **v2** > **base** > **v3**. Almost the MIRROR image of the metric's ranking! DBCV's favourite (v3, 0.377) was your last place.

**Concept: geometry ≠ narrative**

DBCV measures whether the groups are dense and separated (geometry). You measure whether the map TELLS the story of the field (narrative). v3 learned from citations, so it groups *citation communities*, who cites whom: schools, methods, journals, which form beautiful blobs for the metric but do not always coincide with readable TOPICS. v1, with its gentle adjustment, preserves the semantic organisation an expert reads.

## The decisions (yours, as the judge)

- **Production → v1**: you chose it blind TWICE (2026-07-12 and 2026-07-16) and it beats base 14 of 22. Your installation uses local v1; the public version falls back automatically to stock SPECTER (no friction, no manual downloads).
- **v3 is not discarded, it is repurposed**: its "citational" space is exactly the signal the future seed mode needs ("papers similar to THIS one", ResearchRabbit style): there, citation communities ARE what you are looking for.

**Watch out / common mistake**

If we had only looked at the metric, we would have put into production the model the expert reads WORST. If we had only looked at the expert, we would not know WHY v3 groups oddly (nor that it is gold for another task). Machine proposes, expert validates, and the two views together are worth more than either alone.

**In one sentence**

SPECTER's real recipe (citation triplets + hard negatives) applied to YOUR 190,000 papers: 150,885 triplets, 45 min of GPU, v3. The metric crowns it (+0.023 over base, the only clear positive; method > quantity confirmed); your double blind puts it last and crowns v1, a mirror ranking. Double lesson: (1) the training method matters more than the volume of data; (2) geometric quality is not narrative quality, choose the model according to the TASK (v1 for readable maps, v3 as a candidate for citation similarity). "No improvement for this" is also a result, and so is finding its right task.

# Note 45: Semantic zoom: why 10,921 papers fit on screen and 1,772 do not

Froth project · learning Machine Learning by building it

## Glossary in plain words

- **Render**: to draw on the screen. Every dot, every line, every letter you see had to be painted by someone.
- **DOM**: the list of objects the browser keeps for each thing drawn on a page. If you draw 10,000 circles as objects, the browser keeps 10,000 records and revisits them continuously.
- **SVG**: a way of drawing where EACH shape is a DOM object. Convenient (you can click each one) but expensive: thousands of objects drown the browser.
- **WebGL**: a way of drawing that hands the points to the **graphics card** (the GPU). The GPU paints millions of points at once because that is what it is built for. It does not keep a record per point.
- **GPU**: the chip that draws. Your laptop has an RTX 5060. It does one thing very well: the same operation over thousands of pieces of data *in parallel*.
- **Main thread**: the single lane through which the browser attends to EVERYTHING: your clicks, the mouse wheel, the computations, the drawing. If something occupies it, the page stops responding even though it is not "broken".
- **Frame**: one image of the screen. For something to feel fluid you need about 60 per second, that is ≈16 ms for each one.
- **Graph physics**: a simulation where nodes repel each other and edges pull like springs, until the drawing "settles" by itself.
- **Percentile**: if you sort 201 numbers from smallest to largest, the 90th percentile is the value leaving 90% of them below. It lets you say "the big ones" without inventing a number.
- **Centroid**: the middle point of a group. Where we put its name.

## The problem, as it turned up

We raised the paper ceiling of the *Network* view from 400 to 2,500 and 1,772 nodes ended up on screen. I wrote "nothing breaks". Your screenshot said otherwise: a frozen hairball, with no zoom and no panning.

And yet, in that same session, the *Atlas* view drew **10,921 papers** and stayed responsive. Almost six times as many points, and fluid. That contrast is this whole note.

**Concept: drawing is not simulating**

The Network uses **physics**: on every frame it recomputes the forces between nodes and, when you move the mouse, it checks node by node which one is under the cursor. With 1,772 nodes that fills the 16 ms of the frame and no time is left to attend to the wheel. The zoom was not broken: **the lane was occupied**.
The Atlas simulates nothing. The positions were computed beforehand (UMAP, note 08) and saved. It just sends them to the GPU and the GPU paints them. That is why 10,921 cost it less than 1,772 with physics.

**Watch out / common mistake**

**My mistake, spelled out.** When I said "nothing breaks" I had measured two things: how long Python takes to *generate* the HTML, and how long vis.js takes to *settle* the drawing. Both cheap. **I did not measure the interaction**, which is exactly what broke. The lesson, which applies to the whole project: **something being fast to PRODUCE says nothing about whether it is fluid to USE**. They are two different measurements and you have to take both.

## The other half: 201 names do not fit

With the drawing engine fixed, the human problem appears. The Atlas painted the dots with one colour per subtopic and nothing else: pretty, and mute. To know what each blob was you had to go to the side legend and cross-reference colours by eye.

The obvious solution (put each subtopic's name over its centroid) does not work: the LIBS/FTIR corpus has **201 subtopics**. Two hundred and one labels at once are a wall of text, not a map.

### Toy mini-example (before the theory)

Imagine a map of only 5 territories, with these sizes in papers:

| Territory | A | B | C | D | E |
|---|---|---|---|---|---|
| Papers | 100 | 60 | 30 | 12 | 8 |

Now think of Google Maps. From very far away you see "Spain". You zoom in and "Madrid" appears. Closer, "Chamberí". Closer still, the street name. **What is shown changes with distance**, not just the size of the letters. That is semantic zoom.

Applied to our 5 territories: from far away only A; you close in and A and B appear; closer, A, B, C; right on top, all five. The interesting question is **where you put the cuts**, and that is where inventing a number will not do.

**Concept: semantic zoom (Shneiderman, 1996)**

The classic visualisation rule says: *"overview first, then zoom and filter, and details only on demand"*. Translated to Froth: first a few big territories with names, and the small subtopics only when you get close to their area. Never all 201 at once.

## The cuts come from YOUR data (the usual rule)

An invented threshold ("show the ones with more than 50 papers") would work in one corpus and be absurd in another. So the cuts are computed from the **percentiles of the subtopic sizes**

**of that particular corpus**: the 90th percentile (the genuinely big ones), 75th, 50th, and 0 (all of them).

Two of your real corpora, with the numbers that came out:

| Corpus | Papers | Subtopics | Computed cuts (p90/p75/p50/p0) |
|---|---|---|---|
| Beauvoir (your thesis) | 3,867 | 96 | 54 / 35 / 19 / 7 |
| LIBS + FTIR | 10,921 | 201 | 76 / 37 / 20 / 7 |
| Circular economy | 4,946 | 69 | 142 / 69 / 33 / 7 |

Look at the third row: circular economy has FEWER papers than LIBS but its "satellite" cut is nearly double (142 against 76), because its subtopics are fatter and fewer. A fixed number would have left that map either empty or saturated. The corpus decides.

What you see as you close in, measured in the browser over the 10,921 papers:

| Level | Metaphor | Cut | Names on screen |
|---|---|---|---|
| 1 | Satellite | $\geq 76$ | 21 |
| 2 | Regional | $\geq 37$ | 52 |
| 3 | Local | $\geq 20$ | 106 |
| 4 | Street | $\geq 7$ | 201 |

And there is still a second safety net: even when level 4 sends all 201 labels, deck.gl applies a **collision filter** that discards the ones that would overlap, letting the bigger territory win. The 201 can only appear together if they genuinely fit.

**Try it yourself (hands on the keyboard)**

Open the Atlas of a large corpus and do this in order:

1. Without touching anything, count the names you see. There should be few, all of them fat territories.
2. Zoom in two or three steps over one area. You will see new names appear, and only in the area you are looking at.
3. Click a subtopic in the side legend: the camera flies to it.
4. Press **"Whole map"** to return to the general view.

If in step 1 you see a wall of text, the percentile computation is not being applied: tell me and we will look into it.

## The stumble that teaches more than the success

The original plan for "click on a subtopic" was different and more ambitious: recompute a UMAP **with only that subtopic's papers**, to reveal its internal structure, which the global UMAP flattens. The plan said "about 2 seconds, and it can be cached".

Before building it, I measured: **29.2 seconds** for 500 papers, with the real function (`cluster.reduce_to_2d`). Not 2 seconds. Almost thirty.

With 96 subtopics in your thesis corpus, exploring the map would have meant waiting half a minute per click. Unacceptable. So the click does something far humbler: it **flies the camera to the coordinates that already exist**. Zero computation, zero waiting.

Watch out / common mistake

**The figure in the plan was mine, and it was false.** That "~2s" came from no measurement: I wrote it from intuition and it sat in the plan as if it were data. If I had not measured it before programming, we would have built a useless function and you would have discovered it by using it.
Rule: **a figure in a plan is not data until someone measures it**. And the measurements are taken BEFORE building, not after.

Something real is lost, and it is worth saying: the internal sub-structure of each subtopic still cannot be seen. It is noted as a possible future improvement (compute it once, when the corpus is processed, not on click).

## The division of labour that came out of this

Instead of trying to make one view do everything, each one does what it is good at:

| | **Network** | **Atlas** |
|---|---|---|
| Engine | vis.js with physics | deck.gl / WebGL (GPU) |
| Paper ceiling | 400 | the whole corpus |
| What it shows | who resembles whom | the complete territory |
| What it is for | reading the most cited core | getting oriented and choosing where to look |

Lowering the Network from 2,500 to 400 **is not a regression**: it is recognising that physics is useful for seeing relations between a few nodes, and gets in the way of seeing a continent. If 400 feels too few, the number is in the configuration and can be raised without touching code; but only your machine can judge the fluidity, not mine.

In one sentence

Drawing on the GPU (WebGL) scales; simulating physics per frame does not. And when there are more names than room, they are shown by zoom levels, with the cuts computed from the percentiles of YOUR corpus, not from an invented number.

## Further reading

- Shneiderman, B. (1996), *The Eyes Have It*: the origin of "overview first, zoom and filter, then details-on-demand".
- `nightingaledvs.com/how-to-visualize-a-graph-with-a-million-nodes`
- `cambridge-intelligence.com/visualizing-graphs-webgl`
- The code: `froth/atlas.py` (functions `minCountForZoom`, `makeLabels` and `flyTo`).

# Note 46: Why a program is slow to open: import late, and never compute twice

Froth project · learning Machine Learning by building it

## Glossary in plain words

- **Import**: when your program says `import umap`, Python goes to disk, finds that library, reads all of it and holds it in memory. That takes time. And it happens *even if you never use it.*
- **Library**: code other people wrote that you reuse. UMAP, HDBSCAN and PyTorch are libraries.
- **Module level**: the lines flush against the left margin of a `.py` file. They run the moment anyone imports that file, without anyone calling anything.
- **Lazy import**: moving the `import` INSIDE the function that needs it, so the cost is paid the first time you use that function, not when the program opens.
- **Cache**: keeping an already computed result so you do not repeat the work.
- **In-memory cache**: kept while the program is alive. Close the program and it is **gone**.
- **On-disk cache**: kept in a file. It survives closing the program, shutting the machine down, and coming back tomorrow.
- **Cache key**: the label you file the result under. If the label changes, the stored result no longer applies and the work is redone.

## The problem, as it arrived

Batman wrote: *"the wait is far too long"*. Short sentence, no diagnosis. The rule in this project is not to guess: measure.

Every piece of Froth's startup was timed separately. This came out:

| Startup step | Seconds |
|---|---|
| importing `umap` + `hdbscan` | 24.8 |
| importing `torch` + `sentence_transformers` | 19.7 |
| importing `streamlit` | 2.1 |
| loading the SPECTER model | 2.5 |
| DBCV sweep (choosing the granularity) | 2.3 |
| reading the data off disk | 0.2 |
| **TOTAL** | **52.2** |

Concept: What the table teaches

44.5 of the 52.2 seconds are *importing libraries*. And Froth's home screen (the title, the topic dropdown, the box for pasting a new title) **uses none of them**. You were waiting 45 seconds for code that was never going to run.

## First fix: import late

Before, at the very top of `froth/cluster.py`:

```
import umap
import hdbscan
```

After, inside the function that actually uses it:

```
def reduce_to_2d(vectors):
    import umap          # paid here, not when the app opens
    reducer = umap.UMAP(n_components=2, metric="cosine", random_state=42)
    return reducer.fit_transform(vectors)
```

Watch out / common mistake

**The cost does not vanish: it moves.** UMAP still takes its 30 seconds. What changes is *when*. Before, you paid it staring at a blank screen with no way to tell whether the program was alive. Now you pay it while harvesting a new topic, with a progress bar in front of you saying what it is doing. The same wait feels completely different depending on whether you know what you are waiting for.

## But the big one was still hiding

With the imports fixed, cold startup dropped to 8.8 seconds. And yet, opening the real app, it **still took over three and a half minutes**.

The lesson sits right here: **a stopwatch only measures what you point it at**. I had timed the imports, not the whole app. Looking at what the app printed while it worked turned up the real culprit:

```
Naming the subtopics (reading abstracts)...
```

Naming the 44 subtopics. For each one you read its abstracts, pull out candidate phrases, turn them into vectors with SPECTER and keep the one that best represents the group (that is note 40). That is expensive. And it was redone **in full, every single time you opened the application**.

Concept: Why it repeated if it was already "cached"

The code already used `@st.cache_data`, which stores the result **in RAM**. That works beautifully while the app is open: switch tabs and nothing is recomputed. But closing the app frees that memory. Tomorrow you pay the three minutes again.
An in-memory cache protects you from repeating within a session. It does not protect you from repeating *between* sessions. For that you need disk.

## Second fix: save to disk, under the right key

The names now go into a `.json` file. The interesting part is not the saving, it is **what label they are saved under**.

```
key = f"{version}|{mcs}|{years[0]}-{years[1]}"
digest = hashlib.sha1(key.encode()).hexdigest()[:12]
# -> 2_Datos/processed/subtitles_36e4d383cfce.json
```

The label mixes the three things the names depend on:

- **the data version**: harvest more papers and the subtopics change;
- **the granularity**: moving the slider builds different subtopics;
- **the year range**: filtering by year drops papers, and the groups shift.

Change any of the three and the label changes, the stored file is not found, and the work is redone. Change none and it is read from disk instantly.

**Watch out / common mistake**

**This is the classic caching bug, and it is frightening because it fails quietly.** If your key is too narrow (say, only the corpus name), the program shows you yesterday's names on today's data. Nothing breaks, nothing warns you: it simply lies. A cache with a bad key is worse than no cache at all.

**Concept: This does NOT contradict rule 9**

It might look as though storing the result is the opposite of "recompute for every corpus". It is not. It is still computed for *this* corpus, from *its* data. The only thing removed is computing **the same question twice**. A fixed default would mean never measuring; this is measuring once and remembering.

## The result

| | Before | After |
|---|---|---|
| Opening the app until it is usable | >3.5 min | ≈30 s |
| Importing libraries alone | 52.2 s | 8.8 s |
| Does SPECTER load at startup? | yes | no |

**Try it yourself (hands on the keyboard)**

Check it yourself, and learn to measure while you are at it:

1. Open Froth from the desktop shortcut and count until the home screen appears.
2. Now move the granularity slider to a number you have never used (say 7). Notice: *there* it does wait, because that combination is not stored yet.
3. Move it back to 18, then to 7 again. The second time is instant: it is on disk.
4. Look inside `2_Datos\processed\`. There is one `subtitles_...json` per combination you tried. Open one in a text editor: those are your subtopic names.

**In one sentence**

If a program is slow to open, measure before you touch anything: it is almost always doing work that is not needed yet (import it late) or work it already did yesterday (store it on disk, under a key that includes EVERYTHING that can change the answer).